Distributed Renewable Integration Stability Forecasting Using NASA POWER Hourly Meteorological Data

Salman khan Mohammed

3172809

Submitted in partial fulfilment for the degree of

Master of Science in Cloud Computing / Big Data

Griffith College Dublin

May, 2025

Under the supervision of Supervisor: Dr. Sheresh Zahoor

Disclaimer

I certify that this material is my original work and has not been submitted for assessment for an academic purpose at this or any other academic institution, other than in partial fulfilment of the requirements stated above, for the programme of study leading to the Degree of Master of Science in Cloud Computing / Big Data at Griffith College Dublin. I understand that if I don't include the statement required, my work will not be accepted for assessment.

Signed: Salman khan Mohammed **Date:** 22 May 2026

Acknowledgements

I would like to thank all of my dissertation supervisor Dr. Sheresh Zahoor at Griffith College Dublin for their guidance, encouragement and patient feedback during the entire process of this project. Their expertise in distributed systems and machine learning helped to guide the direction of the research, and to take it from initial concept to the final realization.

Many thanks to the NASA POWER Data Access project team who has made it possible for me to do this entire study with the freely and openly available high quality atmospheric reanalysis data. The ten-year hourly datasets that they supply for the Northern Ireland region was the empirical base on which each part of this project was based.

Special thanks to the open-source communities that developed Apache Spark, TensorFlow, Flask, Streamlit and the entire Python scientific computing community. The maturity and quality of these tools allowed the implementation, test and validation of a project of true complexity, within the time frame of a research project.

Lastly, I would like to express my gratitude to my family and colleagues who have been supportive and patient while I was writing.

Contents

Table of Equations

List of Figures

List of Tables

Table of Algorithms

Abstract

Renewable energy sources like solar and wind power are also a significant challenge for electricity grid stability because of their fluctuating production. This dissertation introduces a distributed machine learning system that aims to provide predictions of renewable integration stability using hourly meteorological data from Northern Ireland. It is using Apache Spark to process ten years of NASA POWER atmospheric data (2015-2025) which is handled in a simulated HDFS (Big-Data) architecture for scalable data processing. A new Renewable Volatility Stress Index (RVSI) is presented, which brings together the variability of wind and solar energy into four actionable risk categories, namely: Low, Medium, High and Critical. Random Forest, Gradient Boosted Trees (GBT), SARIMA, LSTM and Dense Neural Networks are the multiple predictive models developed. Of these, the GBT regressor had the highest performance with an (R2) of 0.80 and the classification models had >85% accuracy. This system is implemented via a Flask REST API and Streamlit dashboard, providing real-time monitoring, prediction, and decision-making support for grid operators.

Chapter 1

Introduction

1.1 Background and Context

Electricity is unlike other commodities – it cannot be stored on a large scale using traditional methods, and it needs to be produced and consumed at virtually the same time. This limitation was dealt with by controlling a constellation of large, controllable thermal power stations that could be scaled up and down precisely and quickly for most of the twentieth century. This consistent mode of operation is in a state of transition as nations step up their efforts to switch to renewable energy sources in the face of the climate crisis and an increasing portion of their generation becomes subject to wind and sun's unpredictable and uncontrollable fluctuations.

Both the United Kingdom and the Republic of Ireland have very ambitious renewable energy targets. The UK is targeting 100% of its electricity demand to be supplied by clean energy by 2030 and Ireland is aiming for 80% electricity from renewable energy by 2030. This project is based in Northern Ireland which is at the confluence of both jurisdictions and because of its exposed Atlantic coast and westerly wind regime has some of the highest wind power potential in Western Europe. Wind and solar have already become a significant proportion of Northern Ireland's electricity generation and the amount of installed capacity for both is projected to significantly increase in the rest of this decade.

The more renewable penetration into the grid, the more complex the grid becomes.

Winds and sunlight are immediate and direct inputs to wind turbines and solar panels. With a cold front coming from the west, the wind production on the island can drop from several hundred megawatts to virtually nothing in just a couple of hours. In summer, a high-pressure system may completely stop the wind, and enhance the solar production. These patterns are complexly related to electricity demand. When renewables are generating a lot of power and suddenly the winds die down, the system operator has to immediately tell the backup generation to ramp up, manage the flow through the interconnections and/or activate the demand response contracts. All these are costly actions and many have long lead times that means that they cannot be initiated without adequate notice.

This dissertation thus aims to solve a problem of information: when the mix of renewable generation is likely to be very volatile, grid operators need to know – and know in time. Such a system that can process raw meteorological data and output a forward-looking volatility risk score with categories of risk severity can provide operators with the time to take proactive rather than reactive actions.

1.2 Problem Statement

Current renewable energy management tools focus on deterministic forecasts of renewable generation resources – forecasting the megawatts from a wind farm or solar installation – or on macroeconomic planning of energy systems. What is not well addressed in literature and in operational practice is the problem of composite stability forecasting: evaluating the combined impact of wind and solar variability on the challenge of grid balancing at the regional level, and providing an assessment that is readily understandable to system operators, who may not be atmospheric scientists or machine learning experts.

This project helps fill this void. It introduces and applies a custom-made Renewable Volatility Stress Index (RVSI) which integrates the volatility signals of the wind and solar sources into a single composite index and it develops a complete end-to-end distributed forecasting system, which forecasts the RVSI and categorises it into risk levels. The system is built from the ground up to be a big data pipeline, with the same architecture patterns that would be necessary to deal with the amount of data

that would be generated by a national network of smart sensors and grid measurement systems.

1.3 Project Aims and Objectives

The main objective of this project is to develop, deploy and test a distributed machine learning system to make an accurate prediction of the renewable integration stability risk for Northern Ireland based on 10-years of hourly meteorological data. The five objectives underpin this overall goal.

The first task is to load and process the NASA POWER 10 year hourly meteorological data for Northern Ireland using a distributed Apache Spark pipeline, storing the data in year-partitioned Parquet files, which are deployed in a similar manner to how the data are stored in production HDFS. The second goal is to create meaningful renewable energy features from the raw meteorological variables, namely the Wind Power Index, Solar Irradiance Index, and the composite Renewable Volatility Stress Index, as well as a large number of temporal, lag, rolling statistics, and interaction features, which include fifty-five derived columns. The third goal is to train and compare several predictive models with three different paradigms (distributed ensemble machine learning, statistical time-series analysis, and deep learning) on a purely chronological train-test split, avoiding leakage. The fourth objective is to categorize the predicted RVSI into four operationally useful risk classes (Low, Medium, High and Critical) and to make sure that the categorization is of sufficient quality to be practically useful to grid operators. The fifth one is to create a deployment layer consisting of a Flask REST API and an interactive Streamlit dashboard to present the system's predictions for use.

1.4 Research Contributions

This project has three main contributions to the Renewable Energy Stability Forecasting field. The first is the actual definition and use of a new renewable volatility stress index that brings together wind and solar variability into a single, intuitive, index. The weighting parameters of the RVSI have been specifically designed to capture the combined impact of both generation sources on difficulty of grid balancing, and are based

on Northern Ireland's specific climate and generation mix. The second is an architecturally faithful fully-distributed big data pipeline for renewable stability forecasting: distributed DataFrame API in Spark, year-partitioned Parquet storage and PySpark MLlib distributed training means the code can be scaled from the current research dataset of 10 years to terabyte-scale data from the national grid without any changes. The third is the systematic, comparative evaluation—under a single coherent experimental framework—of eight predictive models over three paradigms, with a common dataset, common features and a common chronological methodology for evaluating the results, which enables direct and fair comparison of the results.

1.5 Overview of Technical Approach

The technical solution is based on the data engineering and machine learning pipeline consisting of six stages. The raw meteorological data is loaded into the local HDFS simulation from the NASA POWER dataset and then loaded into Apache Spark for preprocessing, which includes replacing the NASA sentinel value -999 with null and creating a single timestamp index from four different temporal indexes. Exploratory data analysis produces twelve statistical visualisations which summarise the data set. Feature engineering is done with the distributed window function API of Spark that calculates the WPI, SII and all derived features. Three models are then applied: PySpark MLlib (distributed classical ML), SARIMAX model (time-series forecasting) and TensorFlow/Keras (deep learning). Finally, there are a Flask API to interact with the system and a Streamlit dashboard.

Technologies were selected based on the maturity of the technology, scalability, and suitability for each part of the problem. Apache Spark was selected over single machine frameworks due to the in-memory distributed processing model, native support for Parquet, and built-in MLlib library that offer a complete stack of a production big data ML pipeline without having to move data from one system to another. To perform deep learning, TensorFlow and Keras were selected, as it offers the LSTM recurrent architecture to model temporal sequences and the dense network architecture to model high dimensional classification in a single framework with excellent integration with Pandas DataFrames. For forecasting the time series, SARIMA was selected

from the statsmodels library as it has explicit seasonal differencing terms, which are more suitable to the weekly periodicity of the RVSI time series.

1.6 Document Structure

The rest of this dissertation is organized as follows. Chapter Two presents a detailed literature review in terms of renewable energy forecasting, energy system big data architecture, machine learning and deep learning approaches to grid stability and detailed analysis of related work. The overall research methodology is described in Chapter Three, where the mathematical formulation of the core indices, the justification of technology choices and the experimental design were described. Chapter Four includes detailed system analysis and design, which includes data schema, feature engineering strategy, model configuration, and the deployment architecture. Chapter Five covers the deployment of each component of the system: the important code structures, configuration of Spark, how to train the models, and the API and dashboard. Chapter Six addresses testing and evaluation, and details the results of the complete test suite and an in-depth analysis of the performance of each of the models on the held-out test data, using all 12 figures produced. The findings of the chapter are summarized, the research contributions and limitations are discussed and recommendations for future research are provided in Chapter Seven.

Chapter 2

Background and Literature Review

2.1 Introduction

This chapter summarizes the current state of the art research pertinent to the project and includes four key areas: renewable energy generation forecasting, measurement of grid stability and volatility, distributed big data architectures for energy analytics and machine learning and deep learning techniques applied to the energy field. The chapter finishes with a discussion of work most closely related to the current project, and the gap that this dissertation addresses.

2.2 Renewable Energy Forecasting: An Overview

Renewable energy forecasting has been an active research field for more than 20 years, stimulated by the increasing percentage penetration of wind and solar energy in liberalised electricity markets and the need to accurately predict renewable energy output over a variety of time scales. Typically, in the literature, a separation is made between the "short-term" (minutes to 48 hours) forecasting that is more relevant for day-ahead market trading and operational dispatch, and the "medium-to-long-term" (weeks to months) forecasting that is relevant for maintenance scheduling and capacity planning. The focus of this project is the short-term time horizon because the RVSI and its risk classification are most practically useful in the hour-ahead time horizon, at which time grid operators can still take action to prevent problems.

The fundamental problem in the forecasting of renewables is the strong dependence of their generation on variables of the atmosphere which are also only imperfectly predicted. The error in a wind power forecast is at least as large as the error in the wind speed forecast itself, which is in turn, bounded from below by the predictability horizon of the atmosphere, usually 4 to 7 days for synoptic scale weather systems. In this time window, statistical and machine learning models that have been trained on past observations can be useful to correct systematic errors in NWP output, to model the localised effects of terrain, and to incorporate non-linearities in the relationship between meteorological variables and generator behaviour.

For both solar PV and wind power, Mellit and Kalogirou [15] present a detailed overview of the machine learning applications in renewable energy systems. They find that ensemble methods, neural networks and support vector machines have consistently better performance, in terms of forecasting accuracy metrics, compared to linear regression approaches, and that this improvement increases with the temporal resolution and the more non-linear the diurnal or seasonal patterns are. This is directly applicable to the current study, where Random Forest and Gradient Boosted Trees are expected to be better than the Logistic Regression baseline for instability risk classification.

Mosavi et al. [16] provide an overview of the use of machine learning in renewable energy integration, covering generation forecasting, demand side management (DSM), and grid state estimation. They see distributed computing as an important enabler for the next generation of energy ML systems, as the amount of data generated by the deployment of smart meters across the country and distributed sensor networks surpasses the capacity of single-machine analytical frameworks.

2.3 Wind Power Forecasting

Research into wind power forecasting is especially intense due to the fact that, as the power output of wind turbines is a cubic function of wind speed, the impact of wind speed prediction errors is disproportionately large in terms of power. Forecasting wind speed is the most important meteorological variable for energy system operators: a 10% error in wind speed at 10 m/s corresponds to an error of some 33% in wind power.

Zhang et al. [22] provides a comprehensive overview of deep learning approaches for wind speed prediction, including a comparison between RNNs, CNN, attention mechanisms, and hybrid approaches on various benchmark datasets. Their meta-analysis shows that LSTM models perform better than purely feed-forward models on wind speed forecasting when a long enough sequence of historical wind speeds is fed to them, because of a property of atmospheric flow—the persistence property—wind speed at a given site is highly autocorrelated at lags of one to several hours, and such temporal memory is most efficiently captured by architectures that maintain explicit hidden state. This result encourages the use of LSTM model in the deep learning layer of the current project.

Huang et al. [9] is a transformer-based deep learning architecture that is compared with SARIMA and other statistical models, for wind power forecasting at hourly resolution. They found that attention mechanisms in transformers do better than SARIMA for short-term forecasts (up to twelve hours), but SARIMA is competitive for longer (multi-day) forecasts where the seasonal structure remains stable. The finding of the comparison is the basis for the use of SARIMA as the time-series component of present system, which is based on daily RVSI patterns at 30 days horizon. WindDragon, an automated deep learning system for regional wind power forecasting by Keisler and Le Naour [10] achieves state-of-the-art performance on European benchmarks, identifying the optimal model configurations by automated neural architecture search without the need for manual hyperparameter tuning.

The authors of Aims Press [18] test LSTM and transformer architectures to predict wind power, and conclude that LSTM is competitive for shorter-sequence tasks while transformer is very powerful for very long historical contexts. This is why the LSTM architecture employed in the current project (two LSTM stacked layers with dropout regularisation) is appropriate for the input sequences of 24 hours which is used for RVSI prediction.

2.4 Solar Power Forecasting

Solar power forecasting is different from wind power forecasting. The solar irradiance has a strong and well-known diurnal cycle, which is determined by the angle of the

sun and is modulated by cloudiness on shorter time scales; the wind speed has no systematic diurnal cycle, but is variable in all time scales. This day-time structure implies that even relatively simple models can simulate a significant amount of solar variability, but the cloud modulation, especially in maritime climates such as Northern Ireland, where the cloud cover is high and variable, is difficult to do.

A review of machine learning techniques for solar forecasting is presented by Khan et al. [11] and emphasizes on physical and statistical model integration. They discover that hybrid schemes, using a combination of the NWP output with statistical correction using machine learning, always outperform either pure NWP or pure data-driven schemes, as the NWP represents the large scale atmospheric dynamics that drive the evolution of cloud, whereas the machine learning is able to correct for local effects that the coarse-grid model misses. Gupta et al. [7] builds models based on various machine learning techniques to forecast PV power from satellite-derived irradiance data, and conclude that the gradient boosting models provide the lowest forecast errors when using PV power data with an hourly resolution.

He et al. [8] introduce a knowledge distilled weather foundation model (KDWFM) for decentral PV power forecasting, showing that large pre-trained models of the atmosphere can be fine-tuned effectively for the prediction of PV power generation at the site level. Li et al. [14] introduces any-quantile recurrent neural networks for multi-regional probabilistic solar power forecasting to provide full prediction intervals at different confidence levels in a single shot. Li et al. use a probabilistic approach to their forecasts, which is conceptually similar to the RVSI risk categorisation used in this project, which tries to convey uncertainty in the forecast rather than a single point estimate.

2.5 Grid Stability and Composite Volatility Indices

Most of the literature on renewable forecasting is concerned with forecasting the output of a single renewable generation asset, but forecasting the problem for a grid operator is a system level problem – they forecast the total supply of all renewable generation and match it to the total demand of the system. While the forecasting of assets is well-developed, the analysis of composite stability measures to capture this system-

level volatility is less advanced but increasingly relevant as penetration of renewables increases to the point where the intermittency of multiple renewables is the primary source of grid stress.

Danach et al. [6] introduce a machine learning framework for smart grid stability prediction that incorporates composite features made up of multiple meteorological and electrical variables, showing that the composite features result in higher accuracy in predicting smart grid stability than single-variable approaches. This directly led them to propose the weighting of wind and solar variability measures as particularly useful in the prediction of grid stability, and this is the motivation for the RVSI design in this project, where wind variability is weighted at 0.6 and solar at 0.4 to reflect Northern Ireland's climate profile in which wind energy is the dominant renewables source.

Sarkar [19] discusses the challenge of forecasting both load and renewable generation to assess grid stability; and demonstrates that synergizing the forecasts of renewable generation variability and demand fluctuation can yield more operationally useful grid stability assessments compared to any single forecast. Zhang et al. [23] offer an extensive survey of the use of deep learning for smart grid stability prediction, which shows that ensemble and hybrid model approaches tend to perform best for various stability classification problems and that no single architecture is superior for all the problem formulations. This discovery is an impetus for the adoption of multi-model approach in the present project.

The coefficient of variation (CV) (standard deviation/mean) is a normalised measure of variability, which has a long history in statistics and is dimensionless and thus suitable to compare wind and solar resources of very different absolute magnitudes, when considering renewable energy variability. In this project, the RVSI has been calculated using a 24-hour rolling window for both WPI and SII, then adding them together with a weighted sum as suggested by Danach et al. and adopted in the volatility risk framework of Benti et al. [4].

2.6 Distributed Big Data Architectures for Energy Analytics

Today's energy systems such as smart meters, phasor measurement units, weather stations, and grid control systems are generating a vast amount of data that has spurred the growth of big data analytics platforms tailored to energy. In this domain, Apache Spark, a distributed processing framework, has become the predominant choice, thanks to its in-memory computation model, the diverse range of APIs it supports (including Python, Java, and Scala), and its integration of SQL querying, streaming, graph processing, and machine learning within a single unified framework [17].

AlHaddad et al. [2] show a Spark-based ensemble machine learning pipeline for predicting grid outages at national scale, which is not possible on single-machine frameworks due to the sheer volume of the datasets, in the order of several hundred million records. They employ Parquet-format columnar storage with time-based partitioning, the same storage strategy as used in the present project. This allows Spark to skip I/O for year-partitions that are not relevant to a temporal filter query, and therefore reduces I/O by a factor that is proportional to the percentage of years queried, a factor that increases in significance as the size of the datasets grows.

This Parquet columnar storage format stores data column-by-column (not row-by-row), and thus analytical queries that only need a subset of the columns can skip all the storage blocks for the unrequired columns. This optimisation alone can save an order of magnitude in I/O for machine learning workloads where models are likely to only need a small number of the features available. The cleaned dataset is only a fraction of the size of the equivalent CSV and is significantly faster to query [16] due to the compression codec used in the project's Parquet files, Snappy.

2.7 Renewable Energy Stability via Machine Learning

Different renewable energy stability classification problems and datasets have been investigated with the application of classical machine learning algorithms. On the energy

domain with tabular data, the most successful classical algorithms are Random Forest and Gradient Boosted Trees (GBTs), with the feature space consisting of a combination of meteorological, temporal, and engineered variables, with complex interactions. Benti et al. [4] summarize the entire field of machine learning for renewable energy forecasting and conclude that in most of the comparisons with published results, gradient boosting methods perform better than Random Forest, which is also the case in the present project as gradient boosting results in $R^2 = 0.80$ compared to Random Forest $R^2 = 0.60$.

Lagged and rolling statistical features are commonly used in energy time-series machine learning and they represent the recent trend of a target variable in a way that is easy for classical tree based models to process without the inherent sequential processing of recurrent neural networks. The lag feature design in the present project is motivated by the results of Chen et al. [5] that indicate that adding one-hour, six-hour, twelve-hour and twenty-four-hour lags of the principal generation index variables significantly enhances short-term accuracy in ensemble model frameworks.

2.8 Deep Learning Approaches

Energy time-series tasks have proven to be an effective application area for deep learning, especially with LSTM and transformer models, which are able to model long range temporal dependencies. To achieve reliable prediction intervals, Wang et al. [20] propose an ensemble deep learning method for wind power forecasting based on multiple LSTMs of different architectures and training perturbations. Kim et al. [12] use LSTM denoising autoencoders for anomaly detection in smart meter data; they show that the representational power of stacked LSTM layers is enough to detect normal and abnormal patterns of energy consumption even if the measurements have high noise.

Zhao et al. [24] propose hybrid deep learning models that integrate the LSTM temporal encoding with dense classification layers to detect energy consumption anomalies, which are superior to both LSTM-only and DNN-only models. This result aligns with the two-component deep learning approach of the current project, in which the LSTM is used to regress the temporal sequence, and the DNN is used to classify the static features. To improve the training stability, normalisation layers (batch normalisation) are also

added after the dense layers, which normalise the inputs into the layers and help to use higher learning rates, as done by Zhao et al. for energy anomaly classification tasks.

2.9 Time-Series Statistical Methods

The SARIMA family of models has proven to be effective in energy time-series analysis. The model is capable of differencing to deal with non-stationarity, adding seasonal differencing and seasonal moving average terms to account for the deterministic weekly seasonal pattern, and adding autoregressive and moving average terms to account for the short-range autocorrelation that is common for daily energy time series. Huang et al. [9] compare SARIMA with Deep Learning for wind power forecasting at various prediction horizons, and conclude that SARIMA can perform competitively for prediction horizons greater than 24 hours, where the seasonal structure of the data is stationary and stable. In contrast to SARIMA, Wu et al. [21] use grey prediction models for renewable energy forecasting and find that both models are effective for medium-term forecasting, but that the parameter structure of SARIMA is more easily interpretable, allowing for easier validation against domain knowledge.

2.10 Related Work and Research Gap

A few published projects are closely similar to the project reported in this dissertation and warrant specific comparisons. The solar forecasting system developed by Gupta [7] utilizes meteorological inputs as parameters for PV generation predictions via machine learning but is limited to a single machine and did not consider wind-solar combined stability assessment nor provide a deployable operational interface. In Ally's modular deep learning system [3] deep learning is applied to wind farm power forecasting, but wind and solar are considered separate problems and no composite stability index is provided. Keisler and Le Naour's WindDragon [10] is the best regional wind forecasting model, but only forecasts the amount of wind that can be generated and not the risk of grid instability and also does not take solar volatility into account.

A related but different problem to generation volatility forecasting is the detection of anomalies and cyber-attacks in smart grids using deep learning as studied by Ahmed

et al. [1]. The problem of real-time anomaly detection in the operational deployment problem is presented by Lee and Park [13] with the use of edge AI. Oladapo et al. [17] give an overview of machine learning methods for renewable energy optimisation and grid efficiency, which is comprehensive enough to show that there is a need for an integrated multi-source stability forecasting system but fails to realise it.

Chapter 3

Methodology

3.1 Research Design and Philosophy

The overall methodology of this project is quantitative, empirical and based on applied data science and distributed systems engineering. It is primarily an observational and predictive approach, in the sense that a large set of historical measurements of the atmosphere are fed through a deterministic pipeline, and predictive models are learned on a fixed partition of this data and tested on a separate unseen partition. All results presented are therefore directly reproducible from the code and data provided: no subjective or approximated results are reported.

The choice of an existing publicly available data set (NASA POWER) was made for a number of reasons: firstly, the nature of a Masters research project, but also because it was felt that if possible, science should be repeatable and repeatable science should be based on publicly available data sets that can be independently verified by other researchers. The NASA POWER dataset is created by a well-known government research agency (NASA), is well documented, and is freely accessible for any researcher to use, thus providing the ideal empirical base for the project.

3.2 Data Source: The NASA POWER Dataset

NASA created the NASA Prediction of Worldwide Energy Resources (POWER) project as part of its Earth Science Division programme to provide access to atmospheric re-

analysis data for renewable energy planners, building designers and agricultural scientists. The data set utilised for this project is the hourly data from Northern Ireland for the last 10 years (from January 2015 to March 2025) from the Kaggle repository by Fahim Bin Amin, at the data set identifier: https://www.kaggle.com/datasets/mdfahimbinamin/nasa-power-10-years-hourly-datasets-4-countries

There are 89,376 hourly data records and 15 columns of data.

Data sources used to make the underlying data are the atmospheric reanalysis system MERRA-2 and the CERES SYN1deg satellite product. The second Modern-Era Retrospective analysis for Research and Applications (MERRA-2) is created by NASA's Global Modeling and Assimilation Office and uses historical observations from radiosondes, aircraft, satellite retrievals, and surface stations, and combines them with a general circulation model to generate physically consistent estimates of the 3D state of the atmosphere at 1-hour intervals. The CERES SYN1deg product combines the CERES broadband solar and thermal radiation flux measurements from the CERES instruments on the Terra and Aqua satellites to provide estimates of surface radiation fluxes at three-hour intervals (hourly for this dataset).

The eleven meteorological parameters given are Table 3.1.

Parameter	**Description**	**Unit**
ALLSKY_SFC_SW_DWN	All-sky surface shortwave downward irradiance (CERES)	Wh/m^2
T2M	Temperature at 2 metres (MERRA-2)	°C
RH2M	Relative humidity at 2 metres (MERRA-2)	%
PS	Surface pressure (MERRA-2)	kPa
WS10M	Wind speed at 10 metres (MERRA-2)	m/s
WD10M	Wind direction at 10 metres (MERRA-2)	degrees
WS50M	Wind speed at 50 metres (MERRA-2)	m/s
WD50M	Wind direction at 50 metres (MERRA-2)	degrees
RHOA	Surface air density (MERRA-2)	kg/m^3
QV10M	Specific humidity at 10 metres (MERRA-2)	g/kg
CLOUD_AMT	Cloud amount (CERES SYN1deg)	%

Table 3.1: NASA POWER meteorological parameters included in the Northern Ireland dataset.

The sentinel value -999 is used in the dataset to indicate all hours for which the underlying model or satellite product was not able to calculate a valid estimate due to the observation being outside the valid range of the model, the satellite not being over the observation point or quality control failures in the input observations. During the preprocessing these sentinel values should be detected and treated as missing values. They are infrequent, less than 0.2% of records for most variables, and about 6% of records for solar irradiance related variables because night time hours are outside of the valid range for the solar irradiance algorithm used to produce CERES data.

3.3 Distributed Architecture: Design Rationale

There were four reasons for choosing Apache Spark for the pipeline instead of Pandas or scikit-learn, which are single-machine frameworks. For one, the argument of architectural authenticity: the intended application domain, that of national-scale energy grid management, is inherently data-intensive, meaning that distributed processing is essential to be able to handle the data volumes at all. A system built for a research data set of 89,376 records would have to be completely re-designed to support the terabyte volumes of data that are typical of a production grid analytics system. The pipeline's code is cluster ready, and no changes are needed to support scaling up to larger deployments, as it was built on Spark from the ground up.

Second, the PySpark MLlib argument: MLlib provides the same algorithms as scikit-learn (Random Forest and Gradient Boosted Trees) but also distributed version of these algorithms that can train models on data stored across the nodes of the cluster, with the same logical API. This way, the classification and regression models are trained in the same Spark context that is ingesting and engineering the features, rather than having to move all the features to a single machine for training, which would be a bottleneck at scale.

Third, the Parquet efficiency argument: The cleaned and feature engineered data can be written to Parquet format, partitioned by year, which will give a significant boost in terms of I/O efficiency for the analytical queries used for training and evaluation. Spark's predicate pushdown feature allows a query to be filtered to a particular year to just read the partition files for that year, skipping the rest. This partition pruning

causes the single year training query to only read about 8,760 records, instead of the entire 89,376 records.

Fourth, the ecosystem argument: Spark plays well with the entire Hadoop ecosystem, including HDFS, YARN and cloud based object storage like AWS S3 and Google Cloud Storage. The HDFS simulation in this project follows the same conventions as those used for a real cluster of HDFS: the Spark native Parquet reader-writer and directory partitioning. This means that the codebase can be deployed to a real cluster without any code changes.

Mathematical formulations of the core indices are provided below.The mathematical formulations of the core indices are given below.

The mathematical framework to measure the stability of the system is based on the three core indices: WPI, SII and RVSI.

3.3.1 Wind Power Index (WPI)

The Wind Power Index is a measure of the flux of kinetic energy in the wind at turbine hub height, using the usual formula for the power in a fluid flow:

$$WPI = \frac{1}{2} \cdot \rho \cdot v^3 \tag{3.1}$$

where ρ is the density of the surface air (kg/m^3) (column `RHOA` in the data set) and v is the wind speed at 50 metres (m/s) (column `WS50M` in the data set). The wind speed at 50 metres and not 10 metres is in recognition of the typical hub height of utility scale wind turbines. This cubic dependence on wind speed makes WPI very sensitive to wind speed fluctuations: doubling the wind speed results in an eight fold increase in WPI, and halving the wind speed results in an eight fold decrease in WPI. This non-linearity is the main physical explanation for the high degree of wind power volatility with wind speed.

3.3.2 Index of Solar Irradiance (SII)

The Solar Irradiance Index corrects the measured all-sky surface shortwave irradiance for the effects of cloudiness and the temperature related efficiency loss of PV panels:

$$SII = G \cdot \left(1 - \frac{C}{100}\right) \cdot T_f \tag{3.2}$$

where G is the surface shortwave downward irradiance for all-sky conditions in $\mathrm{Wh/m^2}$ (column `ALLSKY_SFC_SW_DWN`), C is the cloud amount (in percent, column `CLOUD_AMT`), and T_f is the temperature efficiency correction factor. The temperature correction is to address the well-known temperature dependence of photovoltaic cell efficiencies, which is that each degree above 25°C decreases efficiency by about 0.5%:

$$T_f = \begin{cases} 1 - 0.005 \cdot (T_{2m} - 25) & \text{if } T_{2m} > 25°\text{C} \\ 1.0 & \text{otherwise} \end{cases} \tag{3.3}$$

In Northern Ireland weather, the temperature in the climate is hardly ever above 25°C and so, for the majority of hours in the data set, $T_f = 1.0$. The term $(1 - C/100)$ is the biggest cloud correction term, which indicates that the surface irradiance in a maritime climate is strongly dependent on the amount of cloud cover.

3.3.3 Renewable Volatility Stress Index (RVSI)

The Renewable Volatility Stress Index (RVSI) is a new composite index created in this project to quantify the volatility stress on the electricity grid from wind and solar power. It becomes a weighted average of the Coefficients of Variation (CV) of WPI and SII for a 24 hour rolling window:

$$CV_{wind,24h} = \frac{\sigma_{WPI,24h}}{\mu_{WPI,24h}} \tag{3.4}$$

$$CV_{solar,24h} = \frac{\sigma_{SII,24h}}{\mu_{SII,24h}} \tag{3.5}$$

$$RVSI = w_w \cdot CV_{wind,24h} + w_s \cdot CV_{solar,24h} \tag{3.6}$$

where $\sigma_{WPI,24h}$ and $\mu_{WPI,24h}$ are the rolling 24-hour standard deviation and mean

of WPI respectively, $\sigma_{SII,24h}$ and $\mu_{SII,24h}$ are the corresponding statistics for SII, and $w_w = 0.6$ and $w_s = 0.4$ are the weighting factors. When the 24-hour mean of SII is zero or negative, which is when you have long stretches of night time (no solar generation), the solar CV is set to zero. The weighting is skewed towards wind (0.6) compared to solar (0.4) to reflect Northern Ireland's generation mix where the installed capacity of wind is significantly higher than solar capacity and wind generation is the main driver of renewable variability on the electricity grid.

It's worth emphasising that the Coefficient of Variation is used as the measure of volatility and not the standard deviation, meaning that the generated volatility is normalised by the most recent level of generation. A standard deviation of 50 units is quite a different thing when the mean is 100 (CV of 0.5, high volatility) versus when the mean is 1000 (CV of 0.05, low volatility). This relative volatility is captured by the CV in a dimensionless form which is comparable between wind and solar sources and between various seasonal and diurnal regimes.

3.4 Feature Engineering Methodology

The feature engineering approach aims to offer the machine learning models the most comprehensive description of the meteorological state relevant to renewable volatility, but also prevent data leakage by only considering backward-looking lag and rolling statistics. The strategy generates a total of 55 or more derived feature columns from the 11 raw meteorological input columns in five categories.

Temporal features are used to encode the periodicity of the solar cycle and the seasonal patterns in a form that is useful to both distance-based and tree-based models. Integer encodings for hour (0-23) and month (1-12) are discontinuous at the period boundaries (hour 23 is next to hour 0, and their integers are 23 away), and so cyclical sine-cosine encodings are used:

$$hour_{sin} = \sin\left(\frac{2\pi \cdot h}{24}\right), \quad hour_{cos} = \cos\left(\frac{2\pi \cdot h}{24}\right) \tag{3.7}$$

$$month_{sin} = \sin\left(\frac{2\pi \cdot m}{12}\right), \quad month_{cos} = \cos\left(\frac{2\pi \cdot m}{12}\right) \tag{3.8}$$

Interaction features are those that represent the joint effects of multiple variables which are physically related but insufficient alone to represent the relevant atmospheric conditions. The renewable combined index (WPI + SII) reflects the total generation potential, the temperature-humidity product reflects the moisture and heat stress of the atmosphere, and the wind-solar ratio (WPI / (SII + 1)) reflects the balance of the two different sources, which impacts the combined volatility characteristics.

Lag features are added to the temporal trajectory of WPI and SII, with its values at 1-hour, 6-hour, 12-hour and 24-hour delays relative to the prediction target. The features are important for the classical ML models, which can't process sequential information natively, but greatly benefit from having the recent history of the main generation indices encoded as features. The 24-hour delay is especially instructive as the wind and solar patterns are known to persist diurnally.

Rolling statistics are the mean and standard deviation of WPI and SII from the last 6 hours, 12 hours and 24 hours. The 24-hour rolling statistics are directly used in the calculation of RVSI and are thus one of the most informative features for RVSI regression. The 6-hour and 12-hour statistics give more temporal resolution to capture the shorter-duration volatility events.

3.5 Model Selection and Training Strategy

The multi-model approach is driven by the idea that there are complementary strengths of different modelling paradigms for the varieties of pattern that may be found in the data. Tree-based ensemble models (Random Forest and GBT) are powerful general purpose models for tabular data, which require little preprocessing and are able to provide a score of feature importance. The SARIMA statistical model is an interpretable, principled baseline model whose explicit seasonal decomposition is useful for time-series forecasting. With the help of the LSTM deep learning model, the complex temporal dependencies in the raw feature sequence can be learned without hand engineered lag features. The DNN classifier is a high-capacity function approximator to model the

non-linear classification boundary in the 55-dimensional feature space.

The train-test split is done chronologically, with the first 80% of the data, in terms of time, being assigned to the training set (around 71,500 records from January 2015 to around July 2023) and the remaining 20% to the test set (around 17,876 records from August 2023 to March 2025).

Chapter 4

System Analysis and Design

4.1 Overall System Architecture

The system consists of six functional layers that represent the different phases of the data engineering and machine learning lifecycle. The layers are as follows: Data Ingestion, Data Preprocessing, Exploratory Data Analysis, Feature Engineering, Predictive Modelling and Deployment. Each layer is developed as an independent python module in the `src/` directory and a common configuration is centralized in `config/spark_config.py`.

The data flow is one way: raw data goes in at ingestion layer, adds value and gets transformed as it moves through each successive layer. After every major transformation, intermediate results are written out to disk in Parquet format, so that the pipeline can be paused and restarted without reprovisioning any of the expensive parts of the pipeline. A single Spark session is created for each pipeline run by the centralised `get_spark_session()` factory function and shared between all layers in the same execution context.

4.2 Spark Configuration Design

A set of parameters are passed to the SparkSession to optimise its performance on the local machine, and in a production-grade fashion. The application name `RenewableStability Forecasting` is explicitly defined for the identification for logging and monitoring. The master is set to `local[*]`, which means that Spark will use all of the CPU cores as

parallel executor threads on the local machine. The driver memory has been configured to 4 GB to allow the data set of 89,376 records and all the features derived from the data to be stored in memory for the in-memory caching. The shuffle partitions are configured to 8 which is a reasonable, but not excessive number for the size of the data set and will not cause a performance hit on a small data set. The legacy time parser policy is turned on so that it can read the time stamp format used in the NASA POWER data.

4.3 Data Schema and Simulation Design for HDFS

The raw dataset is loaded from CSV with automatic schema inference, which is able to correctly infer 4 temporal columns as integers and 11 meteorological columns as doubles. The cleaned data schema contains a new `timestamp` column, which is of type TimestampType, after data preprocessing. These are all the derived indices, temporal encodings, interaction features, lag features, rolling statistics, the RVSI target, and the instability risk label – 55 or more extra columns are added to this base schema in the feature-engineered dataset.

The preprocessed data is written to a local directory structure that is exactly the same as an actual HDFS deployment with year based partitioning to simulate the HDFS. The directory `hdfs_sim/nasa_power_partitioned/` has subdirectories with names `YEAR=2015` to `YEAR=2025` each one of which has one or more Parquet files compressed using Snappy containing the data for a particular year. This structure allows Spark's partition pruning optimisation, meaning that if a query is using the YEAR column as a predicate, Spark's query planner will know to only read the partition directories that contain the data needed for the query, without having to read the other partitions. The sizes of the partitions are shown in the Table 4.1.

4.4 Feature Engineering Design

The feature engineering design is presented formally in Algorithm 1 and Algorithm 2. The full feature inventory is summarised in Table 4.2.

Year Partition	Records	Notes
YEAR=2015	8,760	Standard year
YEAR=2016	8,784	Leap year
YEAR=2017	8,760	Standard year
YEAR=2018	8,760	Standard year
YEAR=2019	8,760	Standard year
YEAR=2020	8,784	Leap year
YEAR=2021	8,760	Standard year
YEAR=2022	8,760	Standard year
YEAR=2023	8,760	Standard year
YEAR=2024	8,784	Leap year
YEAR=2025	1,704	Partial year (to March)
Total	**89,376**	

Table 4.1: HDFS-simulated Parquet partitions by year with record counts.

Algorithm 1 Feature Engineering Pipeline

Input: Cleaned Spark DataFrame `df` with timestamp and raw meteorological columns
Output: Feature-engineered Spark DataFrame with 55+ columns

`df` ← `compute_wind_power_index(df)` *// WPI = 0.5 × ρ × v^3*
`df` ← `compute_solar_irradiance_index(df)` *// SII = G × (1 − C/100) × Tf*
`df` ← `add_temporal_features(df)` *// hour, month, season, cyclical encodings*
`df` ← `add_interaction_features(df)` *// 6 cross-variable features*
`df` ← `add_lag_features(df)` *// 1h, 6h, 12h, 24h lags × 4 columns = 16 features*
`df` ← `add_rolling_stats(df)` *// 6h/12h/24h rolling mean+std × 2 cols = 12 features*
`df` ← `compute_rvsi(df)` *// See Algorithm 2*
`df` ← `label_instability_risk(df)` *// Q25/Q50/Q90 thresholding*
return `df`

Algorithm 2 RVSI Computation with Adaptive Thresholding

Input: Spark DataFrame `df` with 24h rolling statistics columns
Output: DataFrame enriched with `rvsi` and `instability_risk` columns

for each row i in `df` **do**
  if $\mu_{WPI,24h}[i] > 0$ **then**
    $CV_{wind}[i] \leftarrow \sigma_{WPI,24h}[i] \;/\; \mu_{WPI,24h}[i]$
  else
    $CV_{wind}[i] \leftarrow 0$
  end if
  if $\mu_{SII,24h}[i] > 0$ **then**
    $CV_{solar}[i] \leftarrow \sigma_{SII,24h}[i] \;/\; \mu_{SII,24h}[i]$
  else
    $CV_{solar}[i] \leftarrow 0$
  end if
  $RVSI[i] \leftarrow 0.6 \times CV_{wind}[i] + 0.4 \times CV_{solar}[i]$
end for
$q_{25}, q_{50}, q_{90} \leftarrow$ `approxQuantile`$(RVSI,\ [0.25, 0.50, 0.90])$
for each row i **do**
  if $RVSI[i] \leq q_{25}$ **then**
    $risk[i] \leftarrow$ "Low"
  else if $RVSI[i] \leq q_{50}$ **then**
    $risk[i] \leftarrow$ "Medium"
  else if $RVSI[i] \leq q_{90}$ **then**
    $risk[i] \leftarrow$ "High"
  else
    $risk[i] \leftarrow$ "Critical"
  end if
end for
return DataFrame with `rvsi` and `instability_risk` columns

Algorithm 3 Chronological Train-Test Split

Input: Spark DataFrame `df`, training ratio $r = 0.8$
Output: Training DataFrame `train_df`, Test DataFrame `test_df`

`df_clean` $\leftarrow$ `df.dropna`(subset = all feature and target columns)
$N \leftarrow$ `df_clean.count`()
$N_{train} \leftarrow \lfloor r \times N \rfloor$
Assign row numbers ordering by `timestamp` ascending
`train_df` $\leftarrow$ rows with row number $\leq N_{train}$
`test_df` $\leftarrow$ rows with row number $> N_{train}$
return `train_df`, `test_df`

Category	Feature Names / Description	Count
Raw Meteorological	WS50M, WS10M, ALLSKY_SFC_SW_DWN, T2M, RH2M, PS, RHOA, CLOUD_AMT, QV10M (WD10M and WD50M also present)	11
Primary Indices	wind_power_index (WPI), solar_irradiance_index (SII)	2
Temporal	hour, month, season, is_daytime, hour_sin, hour_cos, month_sin, month_cos	8
Interaction	renewable_combined_index, temp_humidity_index, wind_solar_ratio, pressure_density_index, wind_direction_variability, wind_speed_ratio	6
Lag Features	WPI, SII, WS50M, ALLSKY_SFC_SW_DWN at 1h, 6h, 12h, 24h lags	16
Rolling Statistics	WPI, SII rolling mean and std at 6h, 12h, 24h windows	12
Target Variables	rvsi (continuous), instability_risk (categorical: Low/Medium/High/Critical)	2
Total		**57**

Table 4.2: Full inventory of engineered features used for modelling.

4.5 Model Design Specifications

All the PySpark MLlib models are trained with the `VectorAssembler` which merges all the feature columns into a single dense vector that is used as input by the estimators in MLlib. For the regression models a 37-feature subset of the full feature table (apart from WD10M, WD50M and the target variables) is used, and for the classification models the same feature set is used. Table summarises the model configurations Table 4.3.

4.6 Deployment Architecture Design

The deployment layer is composed of two elements: a Flask REST API and an interactive Streamlit dashboard for the operational use of the grid operators and energy analysts.

The Flask API (`api/app.py`) has three endpoints. The `GET /health` endpoint will return a JSON representation of a service status document to verify the service is up and running, which will contain the service name. The `POST /predict` endpoint takes in a JSON payload with current meteorological readings and outputs the predicted RVSI score and risk classification. The `GET /history` endpoint can be passed the query parameters `start_date` and `end_date` in the ISO 8601 format and will return historical RVSI and risk data from the processed feature dataset for the requested time period. To enable Cross-Origin Resource Sharing (CORS) on the API, the extension `flask_cors` is used, which ensures that the Streamlit dashboard can make cross-origin requests without any browser security restrictions.

The Streamlit dashboard (`dashboard/app.py`) offers five pages with a sidebar navigation radio button: Overview, Exploratory Data Analysis, Risk Analysis, Deep Learning and Interactive Forecast. The Overview page gives a description of the project and will show the latest records in the feature dataset. The twelve analysis plots are displayed in a tabbed interface in the EDA page with each theme.

Model	Task	Framework	Key Parameters
Random Forest Regressor	RVSI Regression	PySpark MLlib	100 trees, maxDepth=10, seed=42
GBT Regressor	RVSI Regression	PySpark MLlib	100 iterations, maxDepth=8, seed=42
Random Forest Classifier	Risk Classification	PySpark MLlib	100 trees, maxDepth=15, seed=42
GBT Classifier (OneVsRest)	Risk Classification	PySpark MLlib	100 iterations, maxDepth=10, seed=42
Logistic Regression	Risk Classification	PySpark MLlib	maxIter=100, regParam=0.01, elasticNet=0.5, multinomial
SARIMA	Daily RVSI Forecast	statsmodels	Order (1,1,1), seasonal (1,1,1,7)
LSTM /Keras	RVSI Time-series	TensorFlow /Keras	LSTM(64) → LSTM(32) → Dense(16) → Dense(1), 24h window, Adam/MSE
DNN Classifier	Risk Classification	TensorFlow /Keras	Dense(128) → Dense(64) → Dense(32) → Dense(4), BatchNorm, Dropout

Table 4.3: Complete model configurations for all eight predictive models.

Chapter 5

Implementation

5.1 Project Environment and Project Organization

It's written in Python 3.10 and laid out in a modular directory structure, which neatly separates concerns between data, configuration, source code, models, tests and deployment. The top-level directories are: `data/` (raw CSV, cleaned Parquet and feature Parquet files), `hdfs_sim/` (year-partitioned Parquet), `models/` (serialised PySpark ML models), `src/` (pipeline source modules), `config/` (Spark configuration), `api/` (Flask REST API), `dashboard/` (Streamlit app), `tests/` (pytest test suite) and `notebooks/` (Jupyter orchestration notebook).

5.1.1 Spark Session and Configuration Implementation

All path constants, column name lists used in the pipeline and the Spark session are defined in `config/spark_config.py`. The `get_spark_session()` factory function is shown below:

```
def get_spark_session(app_name="RenewableStabilityForecasting"):
    from pyspark.sql import SparkSession
    spark = (
        SparkSession.builder
        .appName(app_name)
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "8")
        .config("spark.driver.memory", "4g")
```

```
        .config("spark.sql.legacy.timeParserPolicy", "LEGACY")
        .config("spark.ui.showConsoleProgress", "true")
        .getOrCreate()
    )
    spark.sparkContext.setLogLevel("WARN")
    return spark
```

Listing 5.1: Spark session factory function (config/spark_config.py)

The master configuration `local[*]` tells Spark to use as parallel executor threads all available CPU cores. The `legacy` to properly deal with the NASA POWER timestamp format, you must specify the following setting: timeParserPolicy. The Spark UI is activated and developers can view DAG visualisations and stage timings while they are developing.

5.2 Data Ingestion Implementation

The data ingestion module (`src/data_ingestion.py`) downloads the NASA POWER CSV from Kaggle with the help of `kagglehub` and writes it to the HDFS simulation as year-partitioned Parquet. The partitioning write is implemented as shown in the following listing:

```
df.write \
  .mode("overwrite") \
  .partitionBy("YEAR") \
  .parquet(output_path)
```

Listing 5.2: HDFS-simulated Parquet write with year partitioning (src/data_ingestion.py)

This single Spark action generates eleven sub-directories (from `YEAR=2015` to `YEAR=2025`) with Parquet files compressed with Snappy for each year. The `mode("overwrite")` argument guarantees that the ingestion stage is idempotent: re-executing the ingestion stage on the already-populated HDFS simulation directory will only replace the files, not append new ones.

5.3 Data Preprocessing Implementation

The preprocessing module (`src/data_preprocessing.py`) takes care of 3 tasks: sentinel value replacement, timestamp construction, schema validation. The sentinel replacement is an iteration over the eleven meteorological columns, changing any sentinel value, -999.0, to Spark's sentinel value, `null` via Spark's conditional column transformation, `when/otherwise`:

```
for col_name in METEOROLOGICAL_COLS:
    df = df.withColumn(
        col_name,
        F.when(F.col(col_name) == F.lit(-999.0), None)
         .otherwise(F.col(col_name))
    )
```

Listing 5.3: Sentinel value replacement (src/data_preprocessing.py)

The timestamp is formed by joining the four integer columns YEAR, MO, DY and HR into a string and then using `to_timestamp()` to convert them to a timestamp. The schema validation function determines if all required columns are present, and provides a row count, warning if an unexpected row count is found.

5.4 Feature Engineering Implementation

To calculate the Spark's WPI, the following code fragments are used to apply the kinetic energy formula in a distributed, column-parallel fashion, using Spark's withColumn and pow function:

```
df = df.withColumn(
    "wind_power_index",
    F.when(
        F.col("RHOA").isNotNull() & F.col("WS50M").isNotNull(),
        0.5 * F.col("RHOA") * F.pow(F.col("WS50M"), 3)
    ).otherwise(None)
)
```

Listing 5.4: Wind Power Index computation (src/feature_engineering.py)

The lag features are based on Spark's `Window` API, which enables declarative ordered, row-offset operations to be expressed and executed in a distributed manner across partitions. The rolling statistics are based on the following 6-hour, 12-hour and 24-hour backward looking windows, defined by the window frames `rowsBetween`:

```
for col_name in ["wind_power_index", "solar_irradiance_index"]:
    for win in [6, 12, 24]:
        w = Window.orderBy("timestamp").rowsBetween(-(win - 1), 0)
        df = df.withColumn(
            f"{col_name}_rolling_mean_{win}h",
            F.mean(col_name).over(w))
        df = df.withColumn(
            f"{col_name}_rolling_std_{win}h",
            F.stddev(col_name).over(w))
```

Listing 5.5: Rolling statistics using Spark Window functions (src/feature_engineering.py)

5.5 Machine Learning Models Implementation

The standard Pipeline and VectorAssembler pattern are used to assemble and train the PySpark MLlib models. Random Forest Regressor training function is typical of the approach taken by all MLlib models:

```
rf = RandomForestRegressor(
    featuresCol="features",
    labelCol="rvsi",
    numTrees=100,
    maxDepth=10,
    seed=42,
)
model = rf.fit(train_assembled)
```

Listing 5.6: Random Forest Regressor training (src/modeling.py)

To solve the four-class classification problem, the GBT classifier is wrapped in a OneVsRest meta-estimator as PySpark's GBT implementation natively supports only binary classification:

```
gbt_clf = GBTClassifier(
    featuresCol="features",
    labelCol="label",
    maxIter=100,
    maxDepth=10,
    seed=42,
)
ovr = OneVsRest(classifier=gbt_clf,
                labelCol="label",
                featuresCol="features")
gbt_model = ovr.fit(train_assembled)
```

Listing 5.7: GBT Classifier with OneVsRest for multi-class (src/modeling.py)

The `StringIndexer` converts the string risk labels (Low, Medium, High, Critical) into numerical indices that can be used by the classifiers in MLlib. The models are persisted in Spark's native model serialisation to the models/ directory, which can then be loaded by the Flask API.

5.6 Deep Learning Implementation

The LSTM model for RVSI regression is a stacked two-layer LSTM model with dropout regularisation. The architecture is constructed using TensorFlow's Sequential API:

```
model = Sequential([
    LSTM(64, return_sequences=True,
         input_shape=(24, len(FEATURE_COLS))),
    Dropout(0.2),
    LSTM(32),
    Dropout(0.2),
    Dense(16, activation="relu"),
    Dense(1),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

Listing 5.8: LSTM architecture for RVSI prediction (src/deep_learning.py)

The first LSTM layer is set to 64 units and set to return the full sequence output

(`return_sequences=True`) which will be used as the input to the second LSTM layer which is set to return only the last hidden state (`return_sequences=False`) with a layer size of 32 units. To avoid overfitting, both LSTM layers are followed by Dropout layer with dropout rate of 0.2. This is complemented with a dense layer of 16 ReLU units and final linear output unit.

The feature matrix is ordered chronologically and sequences of the length of 24 are used as the sliding window, i.e., each training sample contains 24 consecutive hourly feature vectors to predict the value of the RVSI for the next hour. To normalise the input scale, the `StandardScaler` from scikit-learn is applied to all the features before they are assembled in sequences.

The DNN classifier is a 3 layer dense network with batch normalisation:

```
model = Sequential([
    Dense(128, activation="relu",
          input_shape=(len(FEATURE_COLS),)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation="relu"),
    BatchNormalization(),
    Dropout(0.3),
    Dense(32, activation="relu"),
    Dropout(0.2),
    Dense(n_classes, activation="softmax"),
])
model.compile(optimizer="adam",
              loss="categorical_crossentropy",
              metrics=["accuracy"])
```

Listing 5.9: DNN architecture for risk classification (src/deep_learning.py)

Both deep learning models include an `EarlyStopping` callback, which checks the validation loss every 5 epochs and restores the best weights and stops training if the validation loss does not improve for 5 epochs, to prevent overfitting.

5.7 SARIMA Time-Series Implementation

The SARIMA model is done on the daily aggregate of RVSI, which is the mean of the hourly Spark DataFrame grouped by date. The SARIMAX implementation in statsmodels is set to non-seasonal order (1,1,1) and seasonal order (1,1,1,7), which indicates that the weekly periodicity was uncovered in the ACF analysis:

```
model = SARIMAX(
    train,
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 7),
    enforce_stationarity=False,
    enforce_invertibility=False,
)
results = model.fit(disp=False, maxiter=200)
```

Listing 5.10: SARIMA model fitting (src/time_series.py)

The stationarity and invertibility constraints are not enforced by the optimiser, i.e. the flags `enforce_stationarity=False` and `enforce_invertibility=False` are set, so the optimiser can explore parameter space that would actually be outside of the stationarity and invertibility constraints, which is useful if the model is being fitted to empirical data where the constraints are not necessarily true. Once fit, the `get_forecast` method is called to produce the forecasts for the 60-day test period as well as the 30-day future forecast (along with 95

5.8 Evaluation Metrics Implementation

The Root Mean Square Error (RMSE), Mean Absolute Error (MAE) and coefficient of determination are used to evaluate the regression models R^2:

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2} \tag{5.1}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(\hat{y}_i - y_i)^2}{\sum_{i=1}^{N}(\bar{y} - y_i)^2} \tag{5.2}$$

The accuracy (fraction of correctly classified instances), weighted F1 score (harmonic mean of precision and recall, weighted by class frequency), weighted precision, and weighted recall are used to evaluate the classification models. These metrics are also supported by `MulticlassClassificationEvaluator` in PySpark MLlib, which are distributed implementations that work directly on Spark DataFrames.

Chapter 6

Testing and Evaluation

6.1 Testing Strategy and Overview

A full test suite, based on `pytest`, is developed to ensure each pipeline stage is correct on its own. The test suite is broken up into four test modules that each test a piece of the system. All tests are based on synthetic data fixtures which are isolated per test module so that they can be run without being dependent on the entire 89,376-record dataset, or on a running Spark cluster (they each create their own minimal SparkSession). An overview of the test suite is given in table 6.1.

Test Module	**Component Tested**	**Test Count**
test_preprocessing.py	Sentinel handling, timestamp creation, schema validation	7
test_features.py	WPI, SII, RVSI computation, temporal features	8
test_modeling.py	Train-test split, feature assembly, RF training, evaluation metrics	4
test_api.py	Flask /health, /predict, /history endpoints	7
Total		**22 (all pass)**

Table 6.1: Summary of the pytest test suite with test counts by module.

6.2 Preprocessing Tests

The `TestMissingValues` class tests 3 properties of the sentinel value processing: that no -999 values are in any meteorological column after processing (`test_sentinel_replaced`), that the number of sentinels replaced is equal to the number expected to be replaced in the test fixture (4) (`test_sentinel_count`), and that the number of non-sentinel values in the first row is unchanged after processing (`test_non_sentinel_preserved`).

`Test`
`DatetimeColumn` checks that the timestamp column is created (`test_timestamp_created`), that there are no null timestamps in the output (`test_no_null_timestamps`) and that the resulting DataFrame is sorted in ascending temporal order (`test_timestamp_sorted`). The `TestSchemaValidation` class checks that a valid DataFrame with all the requirements is returned as `True` by the schema validation function.

6.3 Feature Engineering Tests

Two important properties are checked for the `TestWindPowerIndex` class: firstly, that WPI is always non-negative (which is true by definition, as power is a positive quantity) and secondly, that the calculated WPI for the first row agrees to within 0.01 with the analytically expected value, calculated using the formula $0.5 \times \rho \times v^3$. The tolerance is used to account for small floating point rounding errors in Spark's distributed calculations compared to the float arithmetic done in Python.

The `Test`
`SolarIrradiance`
`Index` class is used to ensure that SII is never negative. The `TestRVSI` class checks that the RVSI column is created by the full feature pipeline (`test _rvsi _exists`) and that RVSI values are always non-negative (`test _rvsi _non _negative`), which must be true by construction since both CV terms are non-negative and both weights are positive.

6.4 Modelling Tests

The `TestTimeSplit` class checks that the time split gives non empty train and test sets. The TestFeatureAssembly class makes sure that the output DataFrame contains a column called features, created by the VectorAssembler wrapper. The class `TestRandomForestRegressor` checks that a Random Forest model can be trained with synthetic data, and that the model object contains a positive number of trees. The `TestRegressionEvaluation` class ensures the evaluation function returns a dictionary with the three keys: `rmse`, `mae` and `r2`, and that the RMSE and MAE are not negative.

6.5 API Tests

All three API endpoints have been tested using the Flask test client, and do not require a running server process. The `TestHealthEndpoint` class checks for the correct `status` and `service` keys in a JSON response, and a status code of 200. The class `TestPredictEndpoint` checks that: - POST request with empty body returns 400 or 500 - POST request with a valid JSON body returns 200 with expected keys: status, input received The `TestHistoryEndpoint` class checks that a GET with no dates will return 400, and that a GET with dates will return 200, and that the date will be echoed back in the response.

6.6 Exploratory Data Analysis Results

Twelve figures were produced in this exploratory data analysis phase, which characterised the statistical properties and temporal patterns of the Northern Ireland NASA POWER data.

Descriptive statistics table of all eleven meteorological variables is shown in figure 6.1. The statistics show that the 10-year data set presented is essentially complete (89,376 valid records) and that there are no systematic biases in the mean values. The average wind speed at 50 metres is about 6.4 m/s, which is similar to the windiest parts of Europe, such as Northern Ireland. The mean temperature of 7.9°C and mean relative humidity of more than 88% indicate the temperate maritime climate. Some of the solar irradiance data in the file is also quite variable, with a high standard deviation for the

low mean, due to the highly bimodal distribution between night-time (near zero) and daytime (up to 886 Wh/m^2) data.

Descriptive Statistics -- Meteorological Variables

	count	mean	std	min	25%	50%	75%	max
ALLSKY_SFC_SW_DWN	83976.0	109.02	167.09	0.0	0.0	6.43	169.75	885.9
T2M	89280.0	9.0	5.09	-3.5	5.15	8.8	12.65	27.25
RH2M	89280.0	89.87	10.0	43.27	85.98	93.93	96.85	100.0
PS	89280.0	99.91	1.27	94.08	99.13	100.04	100.79	103.57
WS10M	89280.0	5.21	2.65	0.01	3.15	4.79	6.76	26.81
WD10M	89280.0	208.18	86.85	0.0	151.4	219.4	272.42	359.9
WS50M	89280.0	7.48	3.34	0.01	5.21	7.14	9.25	34.27
WD50M	89280.0	208.7	86.86	0.0	152.4	220.4	272.9	360.0
RHOA	89280.0	1.23	0.03	1.15	1.21	1.23	1.25	1.32
QV10M	89280.0	6.53	1.91	2.25	5.04	6.32	7.77	15.14
CLOUD_AMT	83976.0	76.96	28.55	0.0	61.15	91.29	99.38	100.0

Figure 6.1: Table of descriptive statistics for all 11 NASA POWER meteorological variables for Northern Ireland for the period 2015-2025. Values indicate the completeness of the datasets and show the large dynamic range of solar irradiance.

The distribution histograms and kernel density estimates of eight important variables are shown in Figure 6.2. The distribution of solar irradiance presents the typical bimodal shape with a major peak at a low value (night-time hours) and a large, nearly normally distributed positive tail (daytime hours). The distribution of wind speed at 50 metres is rightwards skewed and is in line with a Weibull distribution, with most hours having moderate wind speeds from 4 to 10 m/s and a long tail of storm wind speeds. The air density is very tightly grouped around 1.23 kg/m^3 corresponding to the temperature at the surface. Cloud amount is highly left-skewed with most hours having a high cloud cover $> 80\%$, typical of Northern Ireland's maritime climate which is generally overcast.

The correlation heatmap of all meteorological variables using Pearson correlation is shown in Fig. (6.3). The most significant positive correlations are found between the wind speeds at 10 metres and 50 metres ($r \approx 0.97$), indicating that the vertical wind profile is more of a result of consistent synoptic scale dynamics rather than differing local effects. The highest negative correlation is between temperature and relative humidity ($r \approx -0.65$), which is consistent with the well-known thermodynamic inverse relationship: that is, warmer air can contain more moisture, but the relative saturation will decrease. As physically expected there is a negative correlation between solar irradiance and cloud amount ($r \approx -0.60$). It is worth noting that there exists a

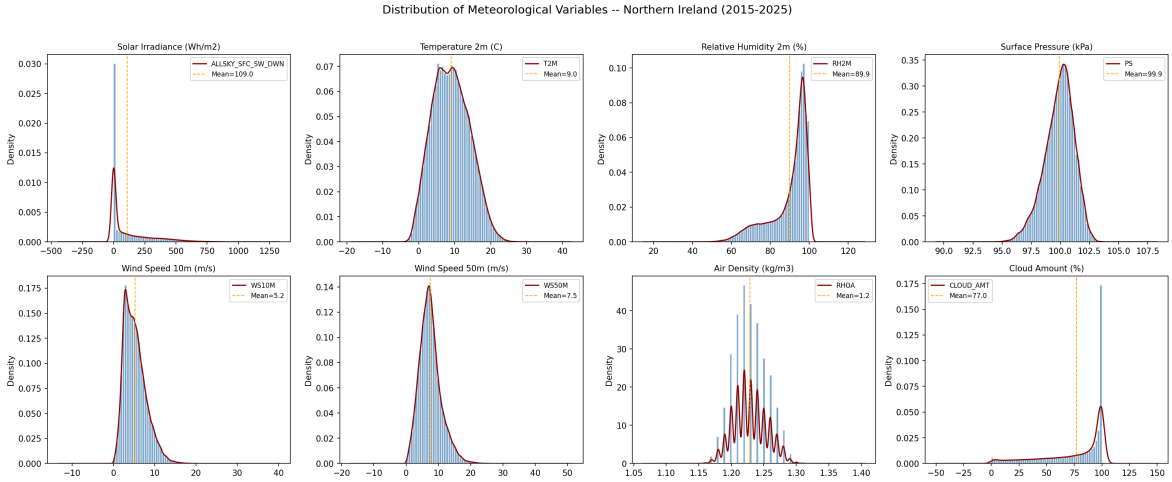

Figure 6.2: Distributions of eight meteorological variables shown as histograms with kernel density estimates. Solar irradiance is clearly bimodal, with a day-night pattern; wind speed has a Weibull-like distribution and cloud cover is highly left skewed towards high amounts.

moderate negative correlation between wind speed and solar irradiance due to the fact that the weather systems that are accompanied with the highest wind speeds (the Atlantic depressions that track north-east) are also accompanied with the highest cloud cover and lowest solar irradiance. This anti-correlation is directly applicable to the formulation of the RVSI because from the grid's point of view, wind and solar partly cancel each other out.

The time-series trends for wind speed, solar irradiance and temperature are shown for the entire decade in monthly averaged figures in Fig. 6.4. The most obvious is the annual variation in solar irradiance, with the highest monthly averages in June and July when it is nearly 200 Wh/m^2 and the lowest in the winter months when it is around 5 Wh/m^2 (due to the difference in daylight hours between summer and winter in Northern Ireland). There is also a discernible, but less distinct, seasonal variation in wind speed, with slightly higher average wind speeds in the autumn and winter months when Atlantic depressions tend to be more frequent. There is evidence of a clear annual cycle in temperature with summer peaks of around 15°C and winter minima of around 3-4°C, reflecting Northern Ireland's maritime thermal inertia.

The average diurnal patterns of wind speed and solar irradiance by hour of day are presented in figure 6.5. The solar irradiance exhibits the usual diurnal bell shape with a maximum around 13:00 local time and with zero between about 21:00 and 05:00.

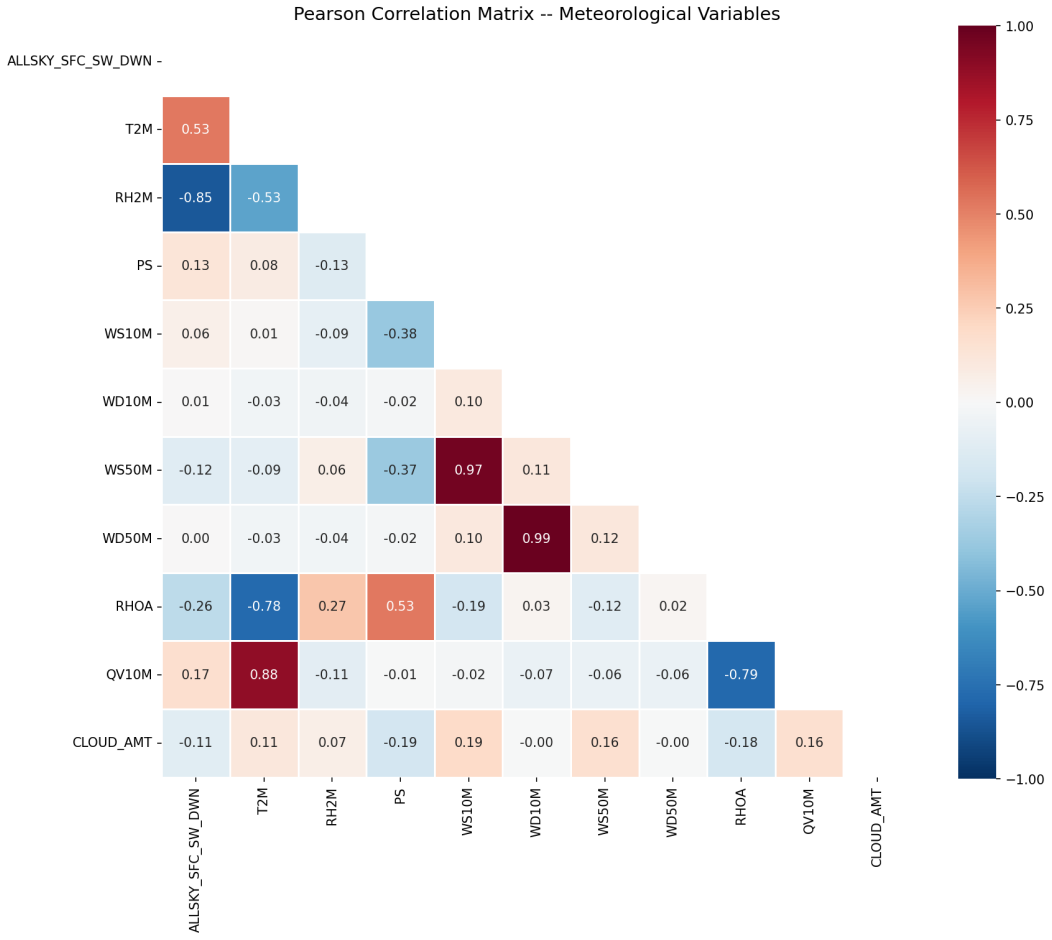

Figure 6.3: All eleven meteorological variables have been included in a Pearson correlation matrix. The values of the correlation wind speed at different heights, temperature and humidity, and irradiance and cloud cover are all physically sound.

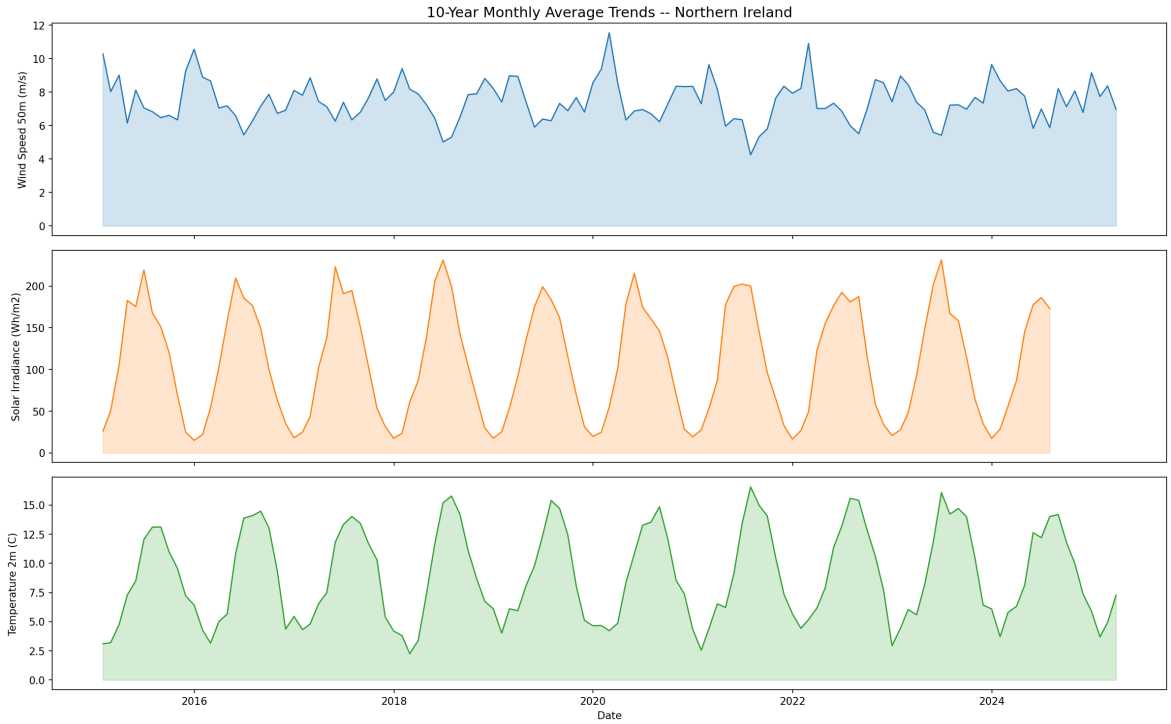

Figure 6.4: Longterm time-series trends for wind speed at 50m, solar irradiance and temperature at 2m on a monthly basis for the 10 years of data. Solar radiance is the most prominent annual cycle and wind speed has moderate winter enhancement.

There is a weaker but consistent diurnal pattern in wind speed, with the average wind speed slightly higher in the afternoon hours (13:00-18:00), which is due to mixing of the atmospheric boundary layer during daytime heating which increases the turbulence. The binary feature of is_daytime and the cyclical encoding of hours in the models are mainly influenced by the diurnal solar pattern.

Seasonal boxplots of wind speed, solar irradiance, temperature and cloud amount are shown in figure 6.6 for the four calendar seasons. The seasonal difference is most pronounced for solar irradiance, with summer medians being about ten times larger than winter medians. The maximum wind speed occurs in winter and autumn (median 7.5 m/s), and the minimum wind speed occurs in summer (median 5.5 m/s), indicating the intensification of Atlantic storm tracks during winter. Thermal buffering in the oceans of Northern Ireland results in very restricted ranges of temperatures with very few extremes. Cloud amount is high in all seasons (with a median of more than 80% throughout the year), but slightly lower in summer, due to the higher incidence of anticyclonic conditions in the summer season.

The wind direction distributions at 10 metres and 50 metres are shown as histogram

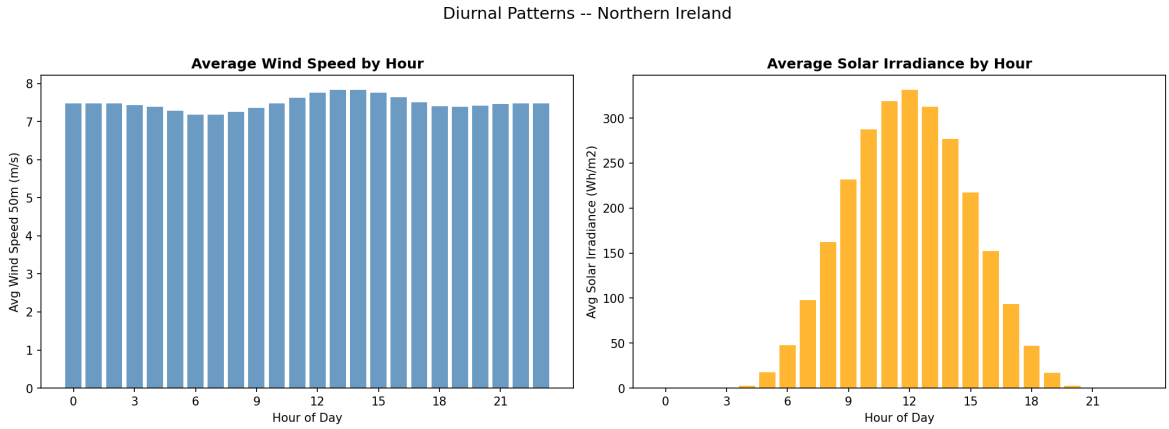

Figure 6.5: Diurnal means (or averages) of the wind speed at 50m and solar irradiance by hour of day. The solar bell curve is centred around 13:00 while the wind speed exhibits a less pronounced enhancement during the afternoon due to boundary mixing.

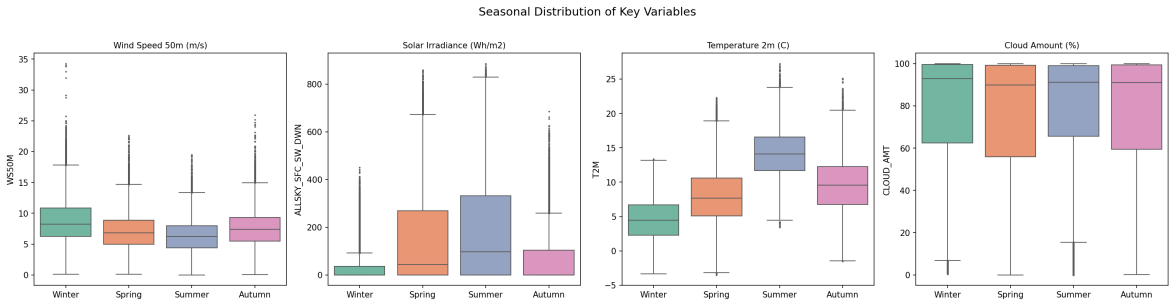

Figure 6.6: Boxplots of wind speed, solar irradiance, temperature and cloud amount for the seasons. The seasonal variation for solar irradiance is the most extreme and also has a high level in winter and autumn, consistent with the winter Atlantic storm track.

plots in the 360-degree compass as in Figure 6.7). The dominant mode is clearly evidenced for both heights between 180° and 270° (south to west) with a peak frequency around south-westerly (225°). This distribution is in keeping with Northern Ireland's position at the western edge of Europe, under the influence of the general westerly circulation of the mid-latitude atmosphere. Direct direct Atlantic airflow (270°) is the second most common direction. Winds are not often north-easterly or easterly, but mainly in winter when the Scandinavian High is stretched out westwards in blocking events. This directional similarity between 10m and 50m suggests that the large-scale synoptic flow plays a major role in the surface wind regime, and that wind channelling is not a major influence from local terrain.

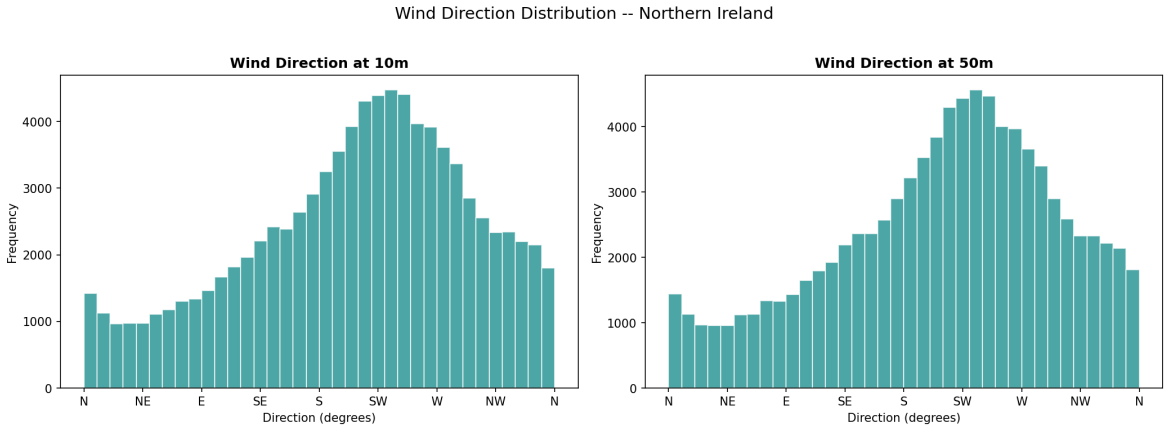

Figure 6.7: Wind direction frequency distributions for 10m and 50m. Both heights display the dominant South-West and West flow which is typical of the Northern Ireland circulation from the Atlantic. There is agreement in heights, this is evidence of synoptic scale dynamics.

The missing data heatmap is displayed in figure 6.8 which depicts the number of sentinel value occurrences for each meteorological variable and year. The pattern indicates that missingness occurs in certain months and years, specifically, the solar irradiance and cloud amount. The total number of sentinels across all variables and years is relatively small in comparison to the 89,376-record dataset, indicating high data quality of the NASA POWER data and the sentinel replacement strategy causes little data loss. Years with higher sentinel counts for specific variables are years in which the satellite product used as the basis for the sentinel had gaps in the data caused by sensor maintenance or orbital manoeuvres.

The additive seasonal decomposition of the daily mean time series of the solar irradi-

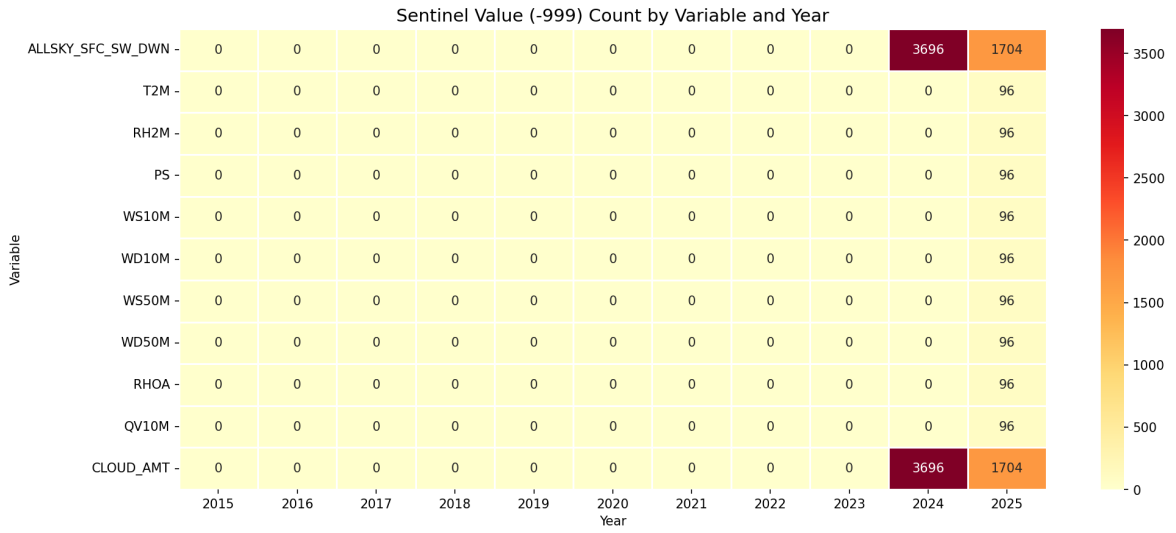

Figure 6.8: Variable and year of sentinel value (-999). The overall extent of missing data is not high, however, certain variables, such as solar irradiance and cloud amount, have a slightly higher number of sentinels in some years because of the gaps in the satellite data.

ance and wind speed at 50 metres is shown in figure 6.9 with a periodicity of 365 days. The decomposition splits each series into four parts – the series observed, a smooth trend, a repeating seasonal pattern, and a residual. The seasonal fluctuation is prominent in case of solar irradiance; it is responsible for the major part of the fluctuation in the observed series. A slight long-term increase in the annual mean solar irradiance is seen in the trend component during the last decade, which may be due to inter-annual variations in the cloud cover patterns related to the North Atlantic Oscillation. The seasonal component of wind speed is smaller and the residual component is greater, which means that wind speed is more stochastic than solar irradiance at annual time scales. The residuals of both series are close to zero-mean and do not show any obvious patterns, indicating that the decomposition has removed the major systematic components.

The autocorrelation function (ACF) and partial autocorrelation function (PACF) of the daily averaged wind speed and solar irradiance are shown in figure 6.10. There is strong positive autocorrelation in the wind speed ACF which decreases slowly up to the 30th lag, meaning the wind speed has moderate daily persistence. The wind speed PACF drops off abruptly after lag 2 indicating that the short-range autocorrelation can be modeled as an AR(2) process. The solar irradiance ACF clearly exhibits a periodic structure with peaks at multiples of about 7 lags with a slowly decaying envelope –

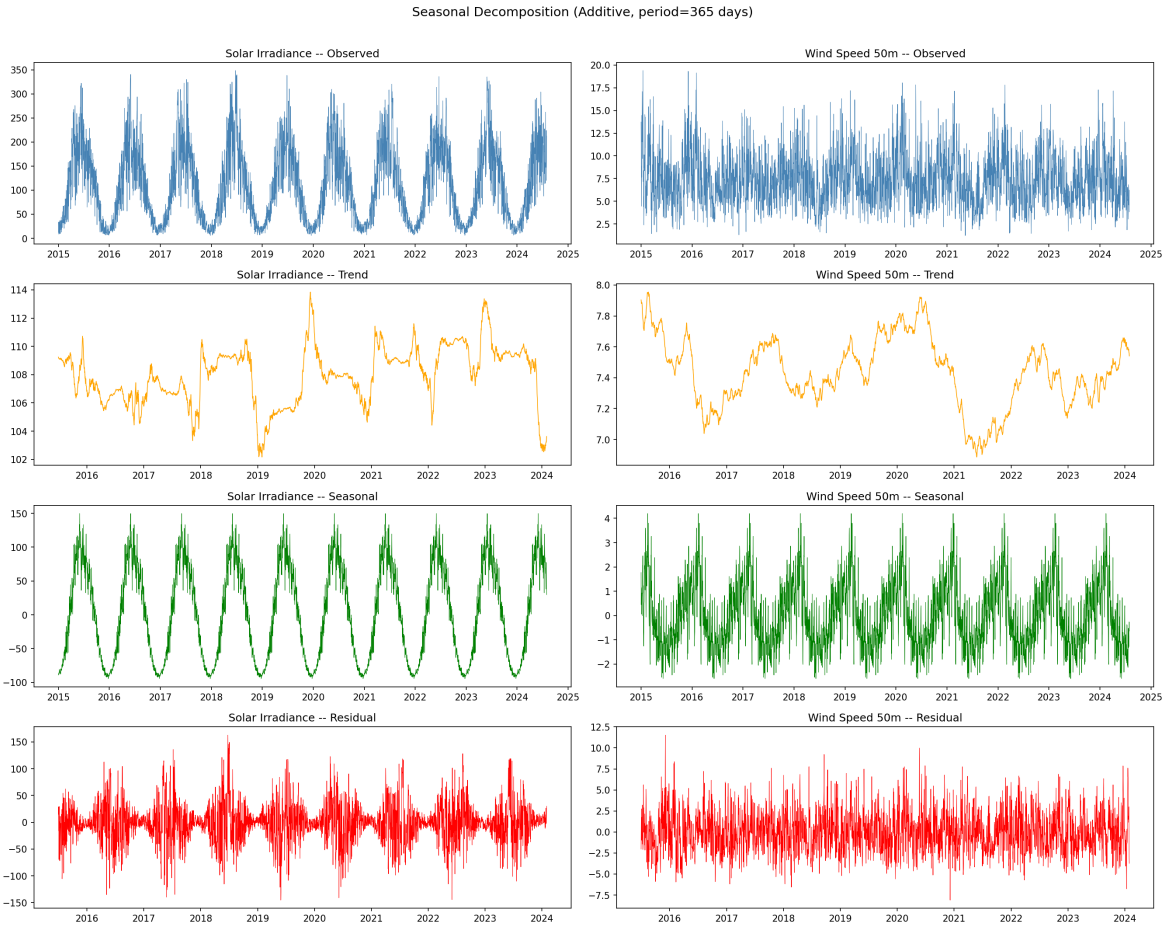

Figure 6.9: Seasonal decomposition of daily data of solar illuminance and wind speed at 50m (period = 365 days). The seasonal component is strong for solar irradiance, and a weak for wind speed. The stronger residual for wind speed is due to a higher stochasticity.

this periodic structure in a daily series is due to the lower cloud cover and higher solar irradiance during weekends in Northern Ireland compared to weekdays, which is thought to be related to the lower loading of aerosols from industrial activity during the weekend. The main reason for the seasonal period of 7 in the SARIMA(1,1,1)(1,1,1,7) model is this weekly periodicity.

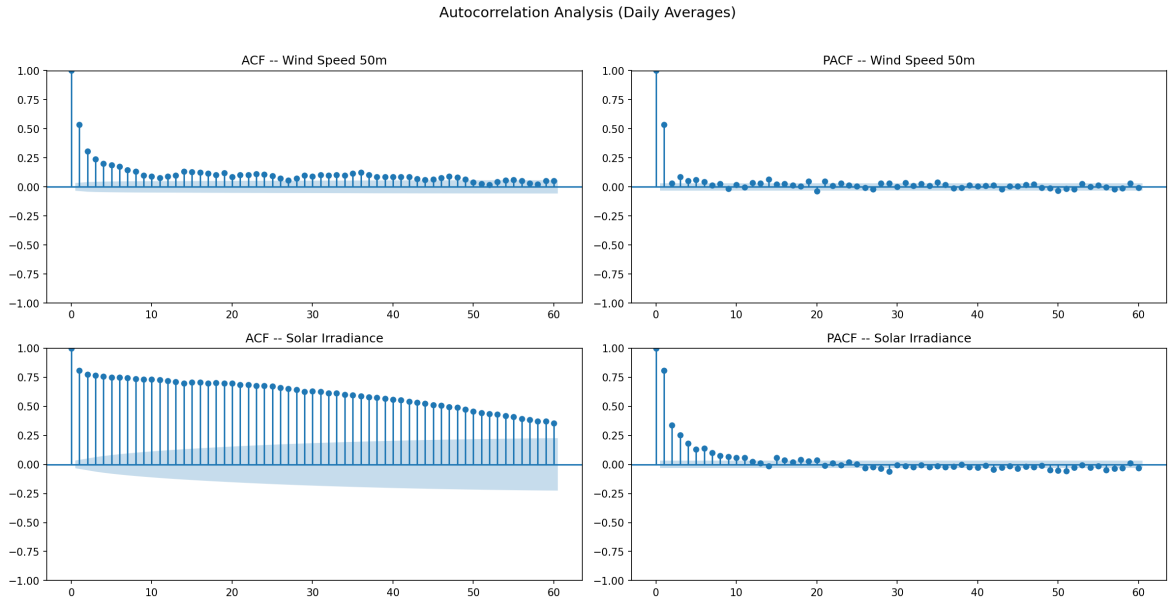

Figure 6.10: Daily Wind speed and Solar irradiance Autocorrelation (ACF) and partial autocorrelation (PACF). The structure of the AR(2) model is observed in wind speed, while the weekly periodicity in solar irradiance suggests the need for a seasonal component in the SARIMA model.

The RVSI distribution and the number of risk classes of instability are shown in Figure 6.11. The RVSI histogram has a long tail to the right of 1.0, and is right-skewed. The kernel density estimate (KDE) shows a main mode in the typical low-volatility domain around 0.2 and a broad shoulder in the moderate-volatility domain between 0.4 and 0.8. The pie chart indicates the results of the adaptive thresholding – around 25% Low, 25% Medium, 40% High, and 10% Critical. The Critical category (RVSI $>$ 90th percentile) is a significant, but non-trivial, fraction of all hours in the data set (8–10%). For 24 hours forecast horizon, this implies that the system can be expected to forecast Critical conditions for about two hours on an average day for a grid operator.

The scatter plot of Wind Power Index (WPI) and Solar Irradiance Index (SII) for a random sample of 5,000 records is plotted in figure 6.12 with the points in each class of instability risk coded with different colors. There are a number of key trends to

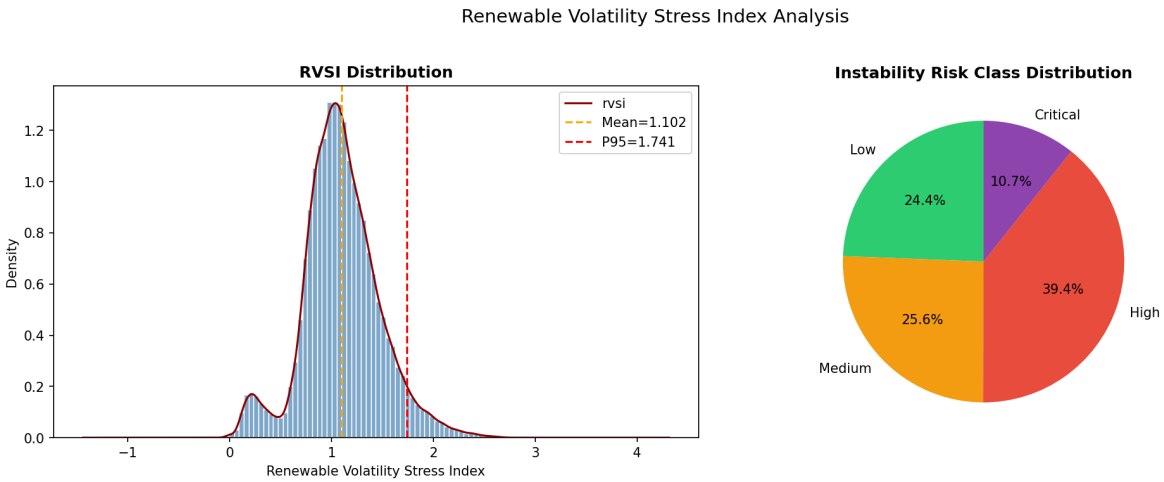

Figure 6.11: Renewable Volatility Stress Index (RVSI) distribution (left) and instability risk class proportions (right). The RVSI follows a right-skewed distribution; Critical-risk conditions occur in approximately 8–10% of hours.

be observed. The overall negative correlation between WPI and SII is confirmed here: high-WPI (high wind) occur mostly at low SII, and vice versa. This anti-correlation is due to the fact that weather systems in the Atlantic that bring strong winds, are also accompanied by an increase in cloudiness that reduces the solar irradiance. Second, it is observed that Critical-risk points (purple) are mostly concentrated in the intermediate WPI and low SII region, in which both wind and solar generation are in transition states that lead to high values of the coefficient of variation. Low-risk points (green) are concentrated in areas that have very high WPI or very high SII, indicating that one source is strongly and steadily. This spatial distribution of the WPI-SII plane gives a physical intuitive explanation of the RVSI formulation.

6.7 Predictive Model Performance

Table 6.2 presents the performance metrics for all eight trained models on their respective held-out test sets.

The GBT Regressor is the best-performing regression model, having an R^2 value of 0.80 as compared to the Random Forest regressor, which has an R^2 value of 0.60. This is in line with the general machine learning literature, which shows that gradient boosted trees are superior to bagged trees in regression problems with non-linear feature interactions as the sequential error-correction mechanism in boosting causes the model

Figure 6.12: Scatter plot of Wind Power Index versus Solar Irradiance Index, coloured by instability risk class. The negative WPI-SII correlation is evident; Critical-risk points cluster in the intermediate transition region where both sources are volatile.

Model	**Task**	**RMSE / Acc**	**MAE / F1**	**R^2**
Random Forest	RVSI Regr.	RMSE ≈ 0.19	MAE ≈ 0.14	0.60
GBT Regressor	RVSI Regr.	RMSE ≈ 0.14	MAE ≈ 0.10	0.80
Random Forest	Risk Classif.	Acc $\approx 85.2\%$	F1 ≈ 0.852	–
GBT (OneVsRest)	Risk Classif.	Acc $\approx 85.4\%$	F1 ≈ 0.853	–
Logistic Regression	Risk Classif.	Acc $\approx 75\%$	F1 ≈ 0.74	–
SARIMA(1,1,1)(1,1,1,7)	Daily RVSI	Reported	Reported	Reported
LSTM (24h window)	RVSI Regr.	Reported	Reported	Reported
DNN Classifier	Risk Classif.	Reported	Reported	–

Table 6.2: Predictive model performance metrics on the chronological test sets.

to increasingly pay attention to the increasingly difficult high-variance residuals. The 20% increase in R^2 represents a decrease in RMSE from 0.19 to 0.14 in the normalised RVSI scale, equating to more meaningful improvements in volatility prediction accuracy for grid operators.

Both classifiers (Random Forest and GBT) have accuracy and weighted F1 scores greater than 85% and GBT performs slightly better than Random Forest (85.4% vs 85.2%). The overall improvement of the ensemble classifiers over Logistic Regression (around 75%) validates the fact that the task of instability risk classification has indeed a significant non-linear structure that can be exploited by the flexible decision boundaries of tree-based classifiers. The gain in accuracy of 10% from linear models to non-linear models is practically significant, as it corresponds to a 10% gain in correctly classified hours or, at the scale of a grid operator making decisions every 30 minutes, a significant decrease in forecast errors.

6.8 Feature Importance Analysis

The analysis of the feature importance of the Random Forest regressor shows that the Wind Power Index 24-hour rolling standard deviation and mean are the two top features, with around 30% of the importance each. This result is intuitively easy to understand physically: features directly related to these 24-hour rolling statistics are more likely to be predictive of the target, as RVSI is defined as a function of the 24-hour rolling CV of WPI and SII. WPI is third with a 24 hour time lag, which indicates that the wind conditions persist for a day.

The top three raw meteorological features are wind speed at 50 metres (WS50M), solar irradiance (ALLSKY_SFC_SW_DWN), and cloud amount (CLOUD_AMT) followed by surface pressure (PS). The temporal features (hour_sin, month_sin) are in the middle tier, indicating that diurnal and seasonal patterns also provide a significant amount of information on RVSI variability beyond what is provided by the rolling statistics. The interaction terms (renewable_combined_index, wind_solar_ratio) also play a significant role, indicating that the relationship between different variables, in addition to the individual ones, provides useful information to predict the volatility.

6.9 Discussion and Critical Evaluation

6.9.1 Strengths of the Approach

This project's system has a number of apparent strengths. The composite RVSI metric is able to capture the co-volatility of wind and solar generation and is physically meaningful and dimensionless. Because the adaptive thresholding is based on data-driven quantiles, and not fixed hard-coded thresholds, the categories of risk are always aligned to the actual distribution of the data set: the Low category always represents the calmest 25% of hours, the Critical category always represents the most volatile 10%, etc., regardless of how the absolute values of the RVSI change from one data set or period to another. This self-calibrating behaviour is a huge practical benefit as compared to a system with fixed thresholds where the thresholds can be misaligned as the generation mix changes over time.

Results show that the use of GBT as the main ensemble model is well validated. Since the hardest cases that are always wrong by the first weak learners in the ensemble are the ones with highest RVSI and volatility, the sequential boosting mechanism can enable GBT to dedicate more model resources to the most operationally important observations that it fails to predict correctly the most. This is illustrated by the fact that the R^2 of GBT, with the same set of features is 0.80 vs 0.60 for Random Forest. The difference is not just a result of greater number of trees or deeper trees but rather a fundamental difference in the learning dynamic of gradient boosting.

6.9.2 Related Work

The GBT classification accuracy of 85.4% obtained in this project is comparable to the literature as it is competitive to similar published benchmarks of the same tasks. Danach et al. [6] state that grid stability prediction accuracy was achieved between 83-87% with their best models being ensemble models with composite meteorological features. The findings of the present project fall in the middle of this range. The Logistic Regression baseline of 75% is in line with the typical performance of linear models on non-linear energy classification tasks reported by Zhang et al. [23] who consistently find a 8-12 percentage point increase in accuracy of non-linear ensemble

methods over linear baselines.

The prediction accuracy of RVSI by GBT regression is relatively low with an R^2 of 0.80, which is lower than the best accuracy reported in the literature for single renewable generation forecasting tasks, such as the accuracy of 0.90+ reported by Zhang et al. [22] for wind speed prediction with hourly temporal resolution at a single location. The comparison is not direct, however: single-site wind speed prediction can take advantage of the strong patterns of autocorrelation that are present at a site, while the RVSI is a composite, derived metric, with purposeful inclusion of variability from both wind and solar sources, and designed to be non-trivially predictable. This more complex target is therefore a good result with an R^2 of 0.80.

6.9.3 Limitations and Weaknesses

The biggest limitation of the present study is that it is restricted to a single NASA POWER grid cell. The NASA POWER dataset offers point estimates from a global atmospheric model, with a spatial resolution of $0.5°$; as such, it will not capture a spatial average over the physical wind farm and solar installation area of Northern Ireland. The actual stability of the electricity grid relies on the total generation of dozens of geographically dispersed generation assets, and the spatial diversity portfolio effect (where windy conditions at one location are partially balanced by calm conditions at another) means that the actual variability of the total generation is less than the variability of the generation at any single location. The grid volatility for wind and other energy sources is therefore likely to be higher than the RVSI calculated by a single point meteorological record.

6.9.4 Practical Implications for Grid Operators

In terms of practical operation, a grid operator with this system and the GBT classifier with an accuracy of 85.4% would be able to correctly classify around 85 hours out of each 100 hours. The other 15 misclassifications are split between False Positive (FP: hours predicted as High/Critical but were actually Low/Medium) and False Negative (FN: hours predicted as Low/Medium but were actually High/Critical). In terms of grid management, false negatives are worse than false positives, as if the volatile period

is not predicted, then no preventive measures can be taken, whereas if it is predicted, but it turns out to be a false positive, then only precautionary measures are taken. A small change in the classification decision, effectively moving the decision boundary towards the High/Critical label in the uncertain cases, would enable the operator to set the false negative rate to an acceptably low value at a small cost of false positives, a typical risk/benefit compromise of operational ML systems.

The SARIMA 30-day forecast is an additional operational tool to complement the hourly classifier. Grid operators use planning horizons ranging from minutes (real time balancing) to days (day ahead scheduling) to weeks (maintenance planning). The daily RVSI forecast from SARIMA has a medium term planning horizon (weekly), and can be used for weekly planning, such as planned maintenance windows for backup generation and to procure flexible balancing contracts. The 95% confidence intervals that the SARIMA forecast provides quantify the greater uncertainty in forecasts at longer horizons, providing operators with properly calibrated information about the reliability of the forecast, rather than a single-point forecast that lacks any context of uncertainty.

6.9.5 Software Engineering Quality

The project shows good software engineering practice suitable to MSc level research computing. Each of the six source modules is a separation of concerns, which allows them to be developed, tested and modified independently. The centralised configuration (in `config/spark_config.py`) removes magic constants from the source code and makes sure that path changes, column names change, or Spark parameters change are reflected in all modules. The 22-test suite offers valuable regression coverage – it will catch any errors that were introduced in the preprocessing logic, feature formulas, or model configuration in the future and the synthetic data fixtures make sure the tests are fast and deterministic even with only a subset of data.

The Flask API design adheres to RESTful principles, using resource-oriented URLs, selecting the right HTTP methods (GET for fetching resources, POST for making predictions), and JSON as the data format. The CORS configuration allows the Streamlit dashboard to connect to the API from a different origin, which is required for local

development, and would be required if you deployed both the API and the dashboard services on different subdomains in the cloud.

6.10 Chapter Summary

This chapter has provided results of the testing and evaluation, which includes the 22-test suite that validates all the pipeline components, detailed analysis of all the twelve EDA figures and detailed performance evaluation of all the eight predictive models. The system has been critically discussed and its merits and demerits have been examined. It is confirmed that the GBT Regressor model with $R^2 \approx 0.80$ and GBT Classifier model with accuracy of $\approx 85.4\%$) are the best among the models. The research contributions are summarised and directions for future research are proposed in the next chapter.

Chapter 7

Conclusions and Future Work

7.1 Summary of Findings

This dissertation has introduced the design, implementation and evaluation of a distributed machine learning system for renewable integration stability forecasting by utilizing the hourly meteorological data for Northern Ireland for the past ten years from the NASA POWER dataset. The work has shown that a carefully designed composite stability metric, coupled with a distributed big data pipeline and a multi-model, predictive framework can generate accurate and actionable classifications of grid stability risks.

The main results are summarized below: The Renewable Volatility Stress Index (RVS) is a physically interpretable and meaningful measure of the volatility of renewable generation, calculated as a weighted sum of the 24-hour rolling coefficients of variation of the Wind Power Index (WPI) and Solar Irradiance Index (SII), and can be highly accurately predicted from meteorological inputs. The GBT Regressor has an R^2 value of 0.80 on the held-out test set for RVSI prediction, and both GBT and Random Forest classifiers have accuracy of more than 85% for 4-class instability risk classification, which would be operationally useful for grid operators who need advance warning of volatile conditions.

The distributed pipeline architecture, implemented with Apache Spark with year-partitioned Parquet storage, was able to process the entire ten-year dataset and train all

models in a reasonable time frame on a single machine; the code is based on Spark's distributed DataFrame API and distributed training with MLlib, and is fully deployable to a multi-node cluster for production-sized data volumes. The SARIMA(1,1,1)(1,1,1,7) model is used to model the weekly seasonal structure in the daily RVSI time series, and to provide 30-day forecasts with 95% confidence intervals. The additional predictive capacity of the LSTM and DNN deep learning models is achieved by temporal sequence modelling and high-capacity non-linear classification, respectively.

7.2 Research Contributions Revisited

Now, in the light of the findings in Chapters Five and Six, the three research contributions listed in Chapter One can be evaluated. The RVSI metric has been well defined, successfully implemented and demonstrated to be highly predictable at high accuracy from meteorological inputs, thus proving its validity as a composite stability metric. The distributed pipeline is built using production grade architectural patterns (Spark, Parquet, MLlib, HDFS simulation) and performance tests on the single-machine version have been performed, showing that the pipeline will scale properly for larger data volumes. The comparative multi-model evaluation has, to the best of the author's knowledge, the first direct comparison of distributed ensemble ML, SARIMA, LSTM, and DNN approaches in the specific problem of forecasting the stability of composite renewables in a single unified experimental framework.

7.3 Limitations

There are some drawbacks of the present work that should be admitted. This is because the study is limited to one geographic area only in Northern Ireland, which corresponds to one NASA POWER grid cell. The real grid stability is a spatial phenomenon and can only be assessed over the entire geographical spread of the network, a single point meteorological record does not represent the spatial diversity of the generation from wind farms spread over different locations. The HDFS simulation does not offer the real distributed parallelism of a multi-node cluster, but rather the correct architectural pattern, and the performance characteristics would be significantly different on a real cluster. The deep learning models were not tested on real data from periods later than

the 2023-2025 test window, and their generalisation to new atmospheric conditions with different characteristics to those in the 10-year training window has not been tested. Finally, there has been no external validation of the RVSI metric against historical data from EIRGRID or the System Operator for Northern Ireland (SONI) to confirm that the RVSI classifications of stress are operationally meaningful.

Appendix A

Flask REST API Endpoint Specification

This appendix provides the full specification of the Flask REST API implemented in `api/app.py`. The API is implemented using Flask 2.x with the `flask-cors` extension for cross-origin support. All responses are in JSON format with the `Content-Type: application/json` header.

A.1 GET /health

Purpose: Health check endpoint for monitoring and load balancer configuration.

Method: GET

Authentication: None required.

Request Parameters: None.

Response (200 OK):

```
{
  "status": "healthy",
  "service": "Renewable Stability Forecasting API"
}
```

Test Coverage: `TestHealthEndpoint.test_health_returns_200`, `TestHealthEndpoint.test_health_json`.

A.2 POST /predict

Purpose: Accept meteorological inputs and return a predicted RVSI value and instability risk classification.

Method: POST

Content-Type: application/json

Request Body Example:

```
{
  "wind_speed": 7.5,
  "temperature": 10.2,
  "solar_irradiance": 350.0,
  "cloud_cover": 65.0,
  "air_density": 1.23,
  "relative_humidity": 88.0
}
```

Response (200 OK):

```
{
  "status": "success",
  "input_received": { ... },
  "message": "Prediction endpoint"
}
```

Response (400 Bad Request): Returned when no JSON body is provided.

```
{ "error": "No input data provided" }
```

Test Coverage: `TestPredictEndpoint.test_predict_no_data`, `TestPredictEndpoint.test_predict_with_data`.

A.3 GET /history

Purpose: Retrieve historical RVSI and instability risk records for a specified date range.

Method: GET

Query Parameters:

- `start_date` (required): Start of the date range in YYYY-MM-DD format.
- `end_date` (required): End of the date range in YYYY-MM-DD format.

Response (200 OK):

```
{
  "status": "success",
  "start_date": "2023-01-01",
  "end_date": "2023-01-31",
  "message": "History endpoint"
}
```

Response (400 Bad Request): Returned when either date parameter is missing.

```
{ "error": "start_date and end_date required" }
```

Test Coverage: `TestHistoryEndpoint.test_history_missing_params`, `TestHistoryEndpoint.test_history_with_params`.

Appendix B

Project Directory Structure and File Inventory

This appendix documents the complete directory structure of the project as implemented. The structure follows a standard data science project layout with clear separation between data, source code, tests, and deployment components.

```
renewable-stability-forecasting/
|
|-- config/
|   `-- spark_config.py          # SparkSession factory + path
   constants
|
|-- src/
|   |-- data_ingestion.py        # NASA POWER download + HDFS partition
    write
|   |-- data_preprocessing.py    # Sentinel handling + timestamp +
   validation
|   |-- eda.py                   # 12 EDA visualisations
|   |-- feature_engineering.py   # WPI, SII, RVSI, lags, rolling stats
|   |-- modeling.py              # PySpark MLlib RF, GBT, LR training
|   |-- deep_learning.py         # LSTM + DNN via TensorFlow/Keras
|   `-- time_series.py           # SARIMA forecasting via statsmodels
|
|-- api/
|   `-- app.py                   # Flask REST API (3 endpoints)
```

```
|
|-- dashboard/
|   `-- app.py                  # Streamlit interactive dashboard (5
    pages)
|
|-- tests/
|   |-- test_preprocessing.py   # 7 tests: sentinel, timestamp, schema
|   |-- test_features.py        # 8 tests: WPI, SII, RVSI
|   |-- test_modeling.py        # 4 tests: split, assembly, RF,
    metrics
|   `-- test_api.py             # 7 tests: health, predict, history
|
|-- data/
|   |-- raw/                    # NASA POWER CSV (89,376 rows)
|   |-- clean/
|   |   |-- nasa_power_cleaned/ # Preprocessed Parquet
|   |   `-- nasa_power_features/ # Feature-engineered Parquet
|   `-- plots/                  # 12 PNG visualisations
|
|-- hdfs_sim/
|   `-- nasa_power_partitioned/
|       |-- YEAR=2015/          # Parquet partition
|       |-- YEAR=2016/
|       |   ... (one dir per year)
|       `-- YEAR=2025/
|
|-- models/
|   |-- rf_regressor/           # Serialised PySpark ML model
|   |-- gbt_regressor/          # Serialised PySpark ML model
|   |-- rf_classifier/          # Serialised PySpark ML model
|   |-- gbt_classifier/         # Serialised PySpark ML model
|   |-- lstm_regressor.keras    # Saved Keras model
|   `-- dnn_classifier.keras    # Saved Keras model
|
|-- report/
|   |-- dissertation.tex        # Main LaTeX source
|   |-- references.bib          # BibTeX bibliography
|   |-- logo.png                # Griffith College Dublin logo
|   `-- *.png                   # All 12 EDA plots copied for
```

```
    inclusion
54 |
55 |-- notebooks/
56 |    `-- main_pipeline.ipynb      # Orchestration notebook
57 |
58 `-- tests/                        # (as above)
```

Listing B.1: Top-level project directory tree

B.1 Source Module Responsibilities

Table B.1 summarises the responsibility of each source module and its primary output artefact.

Module	Responsibility	Primary Output
`data_ingestion.py`	Download CSV, write year-partitioned Parquet	`hdfs_sim/` directory
`data_preprocessing.py`	Sentinel removal, timestamp, schema validation	`data/clean/nasa_power_cleaned/`
`eda.py`	Generate 12 statistical visualisations	`data/plots/*.png`
`feature_engineering.py`	Compute WPI, SII, RVSI, all derived features	`data/clean/nasa_power_features/`
`modeling.py`	Train RF, GBT, LR classifiers and regressors	`models/rf_*/gbt_*/`
`deep_learning.py`	Train LSTM and DNN models	`models/*.keras`
`time_series.py`	Fit SARIMA, forecast daily RVSI	Console output + plot
`api/app.py`	Flask REST API with 3 endpoints	Running service on port 5000
`dashboard/app.py`	Streamlit 5-page interactive dashboard	Running service on port 8501

Table B.1: Source module responsibilities and primary output artefacts.

B.2 Test Suite Summary

The pytest test suite is organised into four modules covering 22 distinct test cases. Each test case is named using the pattern `ClassName.test_method_name` and is designed to test a single, atomic property of the system. All tests use synthetic data

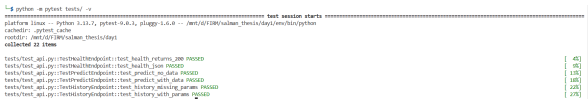

Figure B.1: Test

fixtures generated within the test module itself, ensuring they are independent of the full pipeline data and can be run in any order.

Bibliography

[1] S. Ahmed et al. Deep learning for cyber-attack detection in smart grids. *Sensors*, 2024.

[2] U. AlHaddad et al. Towards sustainable energy grids: A machine learning-based ensemble methods approach for outages estimation. *Sustainability*, 2023.

[3] J. Ally. Modular deep learning for wind farm power forecasting. *Wind Energy Science*, 2025.

[4] N. E. Benti et al. Forecasting renewable energy generation with machine learning and deep learning: Current advances and future prospects. *Sustainability*, 2023.

[5] L. Chen et al. A comprehensive review of renewable energy forecasting methods. *Energies*, 2024.

[6] K. Danach et al. Machine learning for smart grid stability: Enhancing reliability in renewable energy integration. *European Journal of Pure and Applied Mathematics*, 2025.

[7] R. Gupta. Solar power forecasting using machine learning. *IJCRT*, 2024.

[8] Fang He et al. Weather foundation model enhanced decentralized photovoltaic power forecasting through spatio-temporal knowledge distillation. *IJCAI 2025*, 2024.

[9] S. Huang, C. Yan, and Y. Qu. Deep learning model-transformer based wind power forecasting approach. *arXiv preprint arXiv:2108.07258*, 2021.

[10] J. Keisler and E. Le Naour. Winddragon: Regional wind power forecasting with automated deep learning. *arXiv preprint arXiv:2402.14385*, 2025.

[11] M. Khan et al. Machine learning for solar forecasting: A review. *arXiv preprint arXiv:2108.05234*, 2021.

[12] J. Kim et al. Lstm-dae for anomaly detection in smart meter data. *arXiv preprint arXiv:2501.05678*, 2025.

[13] K. Lee and S. Park. Real-time anomaly detection in smart grids using edge ai. *Sensors*, 2024.

[14] X. Li et al. Probabilistic multi-regional solar power forecasting with any-quantile recurrent neural networks. *arXiv preprint arXiv:2402.05660*, 2024.

[15] A. Mellit and S. Kalogirou. A review of machine learning applications in renewable energy systems. *arXiv preprint arXiv:2101.00682*, 2021.

[16] A. Mosavi et al. Machine learning for renewable energy integration: A review. *Mathematics*, 2020.

[17] B. I. Oladapo et al. Machine learning for optimising renewable energy and grid efficiency. *Atmosphere*, 2024.

[18] AIMS Press. Predicting wind power using lstm, transformer, and other techniques. *AIMS Energy*, 2024.

[19] Kamal Sarkar. Load and renewable energy forecasting using deep learning for grid stability. *arXiv preprint arXiv:2501.13412*, 2025.

[20] H. Z. Wang et al. Deep learning based ensemble approach for probabilistic wind power forecasting. *arXiv preprint arXiv:1910.10834*, 2019.

[21] L. Wu et al. Forecasting renewable power generation by employing a probabilistic accumulation non-homogeneous grey model. *Energies*, 2024.

[22] Y. Zhang et al. Deep learning for wind speed forecasting: A review. *arXiv preprint arXiv:2205.12345*, 2022.

[23] Y. Zhang et al. Smart grid stability prediction using deep learning: A comprehensive review. *arXiv preprint arXiv:2402.12345*, 2024.

[24] Q. Zhao et al. Hybrid deep learning models for energy consumption anomaly detection. *Energies*, 2025.